

Anhang 1. ChatGPT-Chatverlauf

Inhaltsverzeichnis

5.2 OCR-Erkennung.....	2
5.3 Teil/ XML-Modellierung (Teil 1)	6
5.3 Teil/ XML-Modellierung (Teil 2)	10
5.4.1 NER (Erstellung von Trainingsdaten).....	14
5.4.2 Relationsextraktion.....	52
5.4.3 Datenabgleich	63
6. SRAPQL	74

5.2 OCR-Erkennung



ChatGPT 5 ▾

[↑ Gemeinsam nutzen](#) ...


Ich habe den folgenden Code. Kannst du ihn bitte korrigieren? Der Code funktioniert nicht, da die TXT-Dateien unverändert bleiben. Er soll einen angegebenen Ordner durchsuchen und in allen darin enthaltenen Unterordnern nach .txt-Dateien suchen. In jeder gefundenen Datei sollen bestimmte Zeichenfolgen ersetzt werden. Die ursprüngliche Formatierung, Zeilenumbrüche und alle anderen Inhalte sollen dabei unverändert bleiben. Verwende dabei nur importierte Bibliotheken (os, re)

```
import os
import re

# Textbereinigungsfunktion
def clean_text(text):
    if not text:
        return ""
    text = re.sub(r'(?<=[A-Za-z])<', 'ch', text)
    text = re.sub(r'(?<=[A-Za-z])>', 'ck', text)
    text = (text.replace("ic", "ich")
            .replace("auc", "auch")
            .replace("nac", "nach")
            .replace("aud", "auch")
            .replace("Jh", "Ich")
            .replace("IJ", "Ich")
            .replace("nad", "nach")
            .replace("»", "")
            .replace("—", " ")
            .replace("|", " ")
            .replace("id", "ich")
            .replace("!", "")
            .replace('/', "")
            .replace('-', "")
            .replace("'", "")
            .replace(',', "")
            .replace("—", " ")
            .replace("„", "")
            .replace("“", ""))
    text = ' '.join(text.split())
    return text.strip()

# Ordnerpfad
folder_path = "ata/5.2_OCR-Erkennung/txt"
```

 Stelle irgendeine Frage


ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

Gemeinsam nutzen ...



```
# Ordnerpfad
folder_path = r"data/5.2_OCR-Erkennung/txt"

# Alle .txt-Dateien im Ordner durchgehen
for filename in os.listdir(folder_path):
    if filename.lower().endswith(".txt"):
        file_path = os.path.join(folder_path, filename)

        # Datei einlesen
        with open(file_path, "r", encoding="utf-8") as f:
            content = f.read()

        # Bereinigung anwenden
        cleaned_content = clean_text(content)

        # Originaldatei überschreiben
        with open(file_path, "w", encoding="utf-8") as f:
            f.write(cleaned_content)
```

Nachgedacht für 51s

Du hast zwei Hauptprobleme im aktuellen Skript:

1. `os.listdir(folder_path)` durchsucht **keine** Unterordner.
2. `text = ' '.join(text.split())` entfernt Zeilenumbrüche und komprimiert Leerzeichen – genau das willst du ja **nicht**.

Hier ist eine korrigierte, einfache Lösung mit `os.walk`, ohne Whitespace-Normalisierung und mit beibehaltenen Zeilenendungen:

```
python 📄 Kopieren ✎ Bearbeiten

import os
import re

# Textbereinigungsfunktion
def clean_text(text):
    if text is None:
        return ''

    # Regex-Ersetzungen
    text = re.sub(r'(?<=[A-Za-z])<', 'ch', text)
    text = re.sub(r'(?<=[A-Za-z])>', 'ck', text)

    # Exakte String-Ersetzungen (⏬) atierung/Zeilenumbrüche bleiben una
    replacements = [
```

+ Stelle irgendeine Frage



SV

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[Gemeinsam nutzen](#) ...


```
# Exakte String-Ersetzungen (Formatierung/Zeile) Kopieren Bearbeiten a
replacements = [
    ("ic)", "ich"),
    ("auc)", "auch"),
    ("nac)", "nach"),
    ("aud)", "auch"),
    ("Jh", "Ich"),
    ("IJ", "Ich"),
    ("nad)", "nach"),
    ("»", " "),
    ("—", " "),
    ("|", " "),
    ("id)", "ich"),
    ("!", " "),
    ("'", " "),
    ("—", " "),
    ("'", " "),
    ("'", " "),
    ("'", " "),
    ("—", " "),
    ("'", " "),
    ("'", " "),
    ("'", " ")
]

for old, new in replacements:
    text = text.replace(old, new)

# KEINE Whitespace-Normalisierung, KEIN strip()
return text

# Ordnerpfad
folder_path = r"data/5.2_OCR-Erkennung/txt"

# Alle .txt-Dateien im Ordner UND in Unterordnern durchgehen
for root, dirs, files in os.walk(folder_path):
    for filename in files:
        if filename.lower().endswith(".txt"):
            file_path = os.path.join(root, filename)

            # Zeilenenden beibehalten: newline="" verhindert Übersetzung
            with open(file_path, "r", encoding="utf-8", newline="") as f:
                original = f.read()

            cleaned = clean_text(original)

            # Nur schreiben, wenn etwas geändert hat (schont Zeitst
            if cleaned != original:
                with open(file_path, "w", encoding="utf-8", newline="")
```

+ Stelle irgendeine Frage



SV

 ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



```

    \_ , \_ ),
    ( " " , \_ ),
    ( \_ , \_ ),
    ( " _ " , " " ),
    ( " " , " " ),
    ( " " , " " ),
]

for old, new in replacements:
    text = text.replace(old, new)

# KEINE Whitespace-Normalisierung, KEIN strip()
return text

Ordnungpfad
der_path = r"data/5.2_OCR-Erkennung/txt"

Alle .txt-Dateien im Ordner UND in Unterordnern durchgehen
root, dirs, files = os.walk(folder_path):
for filename in files:
    if filename.lower().endswith(".txt"):
        file_path = os.path.join(root, filename)

        # Zeilenenden beibehalten: newline="" verhindert Übersetzung
        with open(file_path, "r", encoding="utf-8", newline="") as f:
            original = f.read()

        cleaned = clean_text(original)

        # Nur schreiben, wenn sich etwas geändert hat (schont Zeitstempel)
        if cleaned != original:
            with open(file_path, "w", encoding="utf-8", newline="") as f:
                f.write(cleaned)

            # Optionales Feedback:
            print(f"Bereinigt: {file_path}")

```

Falls deine Dateien nicht UTF-8 sind, kannst du testweise beim Lesen

`errors="ignore"` hinzufügen. Ansonsten sollte das so funktionieren: es traversiert alle Unterordner, ersetzt nur die gewünschten Zeichenfolgen und lässt Formatierung sowie Zeilenumbrüche in Ruhe.



+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

5.3 Teil/ XML-Modellierung (Teil 1)

ChatGPT 5

Gemeinsam nutzen

...

Erstelle Python-Code, der für jede .txt-Datei in einem angegebenen Ordner ein <pb>-Element in eine XML-Struktur einfügt. Dieses <pb>-Element soll das Attribut n enthalten, dessen Wert dem Dateinamen ohne die Endung .txt entspricht. Ich arbeite mit xml.etree.ElementTree

Alles klar – ich schreibe dir einen einfachen Python-Code mit `xml.etree.ElementTree`, der alle `.txt`-Dateien in einem Ordner durchläuft und für jede Datei ein `<pb>`-Element mit dem Attribut `n="Dateiname"` in eine XML-Struktur einfügt.

Hier ein funktionsfähiges Beispiel:

python

Kopieren Bearbeiten

```

import os
import xml.etree.ElementTree as ET

# Pfad zum Ordner mit den TXT-Dateien
folder_path = r"data/5.2_OCR-Erkennung/txt"

# Root-Element erstellen
root = ET.Element("root")

# Alle TXT-Dateien im Ordner durchgehen
for filename in os.listdir(folder_path):
    if filename.lower().endswith(".txt"):
        # Dateiname ohne .txt
        base_name = os.path.splitext(filename)[0]

        # pb-Element mit Attribut n erstellen
        pb_element = ET.Element("pb")
        pb_element.set("n", base_name)

        # Element dem Root hinzufügen
        root.append(pb_element)

# XML-Baum erstellen
tree = ET.ElementTree(root)

# XML-Datei speichern (UTF-8 ohne Order Mark)
output_file = os.path.join(folder_path, "output.xml")

```

+

Stelle irgendeine Frage

SV

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[↑ Gemeinsam nutzen](#) ...


```

if filename.lower().endswith(".txt"):
    # Dateiname ohne .txt
    base_name = os.path.splitext(filename)[0]

    # pb-Element mit Attribut n erstellen
    pb_element = ET.Element("pb")
    pb_element.set("n", base_name)

    # Element dem Root hinzufügen
    root.append(pb_element)

# XML-Baum erstellen
tree = ET.ElementTree(root)

# XML-Datei speichern (UTF-8 ohne Byte Order Mark)
output_file = os.path.join(folder_path, "output.xml")
tree.write(output_file, encoding="utf-8", xml_declaration=True)

print(f"XML-Datei wurde erstellt: {output_file}")

```

◆ So funktioniert es

- `os.path.splitext(filename)[0]` entfernt `.txt`
- `ET.Element("pb")` erzeugt das `<pb>`-Element
- `pb_element.set("n", base_name)` setzt das Attribut `n`
- `tree.write()` speichert die XML-Datei

◆ Beispielausgabe bei Dateien `seite1.txt`, `seite2.txt`:

```

xml
<?xml version='1.0' encoding='utf-8'?>
<root>
  <pb n="seite1" />
  <pb n="seite2" />
</root>

```

Wenn du willst, kann ich dir den Code so erweitern, dass er **die Inhalte der TXT-Dateien** direkt in die XML einbettet und das `<pb>`-Element als Trenner dazwischen setzt.


 Stelle irgendeine Frage


SV

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

Gemeinsam nutzen ...



Schreibe Python-Code, der einen Text verarbeitet und anhand von zwei aufeinanderfolgenden Zeilenumbrüchen in einzelne Absätze aufteilt. Danach erstelle ein neues `<p>`-Element in `div_el` und setze den Absatztext als Inhalt (`.text`) dieses `<p>`-Elements.

Klar – hier ist ein Beispielcode, der genau das macht:

Er nimmt einen Text, teilt ihn bei **zwei aufeinanderfolgenden Zeilenumbrüchen** (`\n\n`) in Absätze und erstellt zu jedem Absatz ein `<p>` -Element innerhalb eines vorhandenen `div_el` mit `xml.etree.ElementTree`.

```
python 📄 Kopieren ✎ Bearbeiten

import xml.etree.ElementTree as ET

# Beispieltext
text = """Dies ist der erste Absatz.

Dies ist der zweite Absatz.

Und hier der dritte Absatz."""

# Beispiel: vorhandenes div-Element
root = ET.Element("root")
div_el = ET.SubElement(root, "div")

# Text in Absätze aufteilen (zwei Zeilenumbrüche = Absatztrenner)
paragraphs = text.split("\n\n")

# Für jeden Absatz ein <p>-Element hinzufügen
for para in paragraphs:
    para = para.strip()
    if para: # Leere Absätze ignorieren
        p_el = ET.SubElement(div_el, "p")
        p_el.text = para

# Ausgabe als XML
tree = ET.ElementTree(root)
tree.write("output.xml", encoding="utf-8", xml_declaration=True)
```

Beispielausgabe (`output.xml`):

```
xml ↓ 📄 Kopieren ✎ Bearbeiten

+ Stelle irgendeine Frage 🔊 🔇
```

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

SV



ChatGPT 5

[↑ Gemeinsam nutzen](#) ...


```
# Beispiel: vorhandenes div-Element
root = ET.Element("root")
div_el = ET.SubElement(root, "div")

# Text in Absätze aufteilen (zwei Zeilenumbrüche = Absatztrenner)
paragraphs = text.split("\n\n")

# Für jeden Absatz ein <p>-Element hinzufügen
for para in paragraphs:
    para = para.strip()
    if para: # Leere Absätze ignorieren
        p_el = ET.SubElement(div_el, "p")
        p_el.text = para

# Ausgabe als XML
tree = ET.ElementTree(root)
tree.write("output.xml", encoding="utf-8", xml_declaration=True)
```

Beispielausgabe (output.xml):

```
xml

<?xml version='1.0' encoding='utf-8'?>
<root>
  <div>
    <p>Dies ist der erste Absatz.</p>
    <p>Dies ist der zweite Absatz.</p>
    <p>Und hier der dritte Absatz.</p>
  </div>
</root>
```

Wenn dein Text auch **Windows-Zeilenumbrüche** (`\r\n`) oder gemischte Zeilenumbrüche enthalten kann, würde ich vorher mit `text.replace("\r\n", "\n")` vereinheitlichen, damit die Absatztrennung zuverlässig funktioniert.

Willst du, dass ich dir direkt eine **Version** schreibe, die den Text aus einer **.txt-Datei** liest und das XML erzeugt?


 Stelle irgendeine Frage

 ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

5.3 Tei/ XML-Modellierung (Teil 2)

The image displays two sequential screenshots of a web-based chat application, likely ChatGPT, used for developing a Python script to process XML files.

Screenshot 1 (Top):

- Header:** "ChatGPT 4o" followed by a dropdown arrow. On the right, there are links for "Gemeinsam nutzen" and a menu icon.
- Code Block:** A light gray box containing Python code:

```
Ich habe folgender Code geschrieben für eine XML-Datei. Schreib mir bitte einen neuen. Er soll einen Ordner durchsuchen und auf alle XML-Dateien meinen Code anwenden. Und die Dateien überschreiben  
  
import spacy  
import os  
import re  
from lxml import etree as ET  
  
# spaCy-Modell importieren  
ner_model = spacy.load("ML-spaCy/output/model-best")  
  
# XML-Dateien importieren  
file_path = "data/5.3_TEI-Modellierung/1823_Bonatz_Johanna.xml"  
  
# Namespace-Map für TEI  
ns = {'tei': 'http://www.tei-c.org/ns/1.0'}  
  
# Entität-Typen zu Tags zuordnen  
tag_map = {"PER": "persName", "LOC": "placeName", "DATE": "date"}  
  
# XML-Datei laden  
parser = ET.XMLParser(remove_blank_text=True)  
tree = ET.parse(file_path, parser)  
root = tree.getroot()  
  
# Alle <p>-Elemente finden  
p_elems = root.xpath('//*[tei:text//tei:p]', namespaces=ns)  
  
# Jeden Absatz bearbeiten
```
- Input Field:** A white bar at the bottom with a plus icon, the placeholder text "Stelle irgendeine Frage", a microphone icon, and a send icon.
- Footer:** Small text stating "ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe Cookie-Voreinstellungen."

Screenshot 2 (Bottom):

- Header:** Identical to the first screenshot.
- Code Block:** Continues the Python code from the previous block:

```
# Jeden Absatz bearbeiten  
for p_elem in p_elems:  
    # Leere Absätze überspringen  
    if p_elem.text is None and len(p_elem) == 0:  
        continue  
  
    # Text aus dem Absatz holen und bereinigen  
    full_text = "".join(p_elem.itertext())  
    cleaned_text = re.sub(r'(?<=|d)\.(?=[s$])', '', full_text).strip()  
  
    # Absatz leeren  
    p_elem.clear()  
  
    # In Sätze aufteilen (am Punkt + Leerzeichen)  
    sentences = [s.strip() for s in cleaned_text.split('.') if s.strip()]  
  
    for sentence in sentences:  
        doc = ner_model(sentence) # Named-Entity-Recognition anwenden  
        ents = [ent for ent in doc.ents if ent.label_ in tag_map]  
  
        # Wenn keine Entitäten gefunden, Satz einfach wieder einfügen  
        if not ents:  
            p_elem.text = (p_elem.text or "") + sentence + ". "  
            continue  
  
        # Neues <s>-Element erstellen  
        s_elem = ET.Element("s")  
        last_idx = 0  
  
        for ent in ents:
```
- Input Field:** Identical to the first screenshot.
- Footer:** Identical to the first screenshot.

ChatGPT 4o

Gemeinsam nutzen



```
# Neues <s>-Element erstellen
s_elem = ET.Element("s")
last_idx = 0

for ent in ents:
    # Text vor der Entität einfügen
    if ent.start_char > last_idx:
        chunk = sentence[last_idx:ent.start_char]
        if len(s_elem) == 0:
            s_elem.text = chunk
        else:
            s_elem[-1].tail = chunk

    # Entität als XML-Element einfügen
    tag = tag_map[ent.label_]
    ent_elem = ET.Element(tag)
    ent_elem.text = ent.text
    s_elem.append(ent_elem)

    last_idx = ent.end_char

# Rest vom Satz + Punkt einfügen
remaining = sentence[last_idx:] + ". "
if len(s_elem) == 0:
    s_elem.text = remaining
else:
    s_elem[-1].tail = remaining

# Satz-Element anhängen
p_elem.append(s_elem)
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 4o

Gemeinsam nutzen



```
# Neue Datei speichern
tree.write("ausgabe.xml", encoding='utf-8', pretty_print=True,
xml_declaration=True)
print("Datei gespeichert: ausgabe.xml")
```

Hier ist dein überarbeiteter Code. Er durchsucht rekursiv einen angegebenen Ordner nach `.xml`-Dateien, wendet dein NER-Tagging auf jede Datei an und überschreibt die Originaldateien direkt:

```
python
import spacy
import os
import re
from lxml import etree as ET

# spaCy-Modell laden
ner_model = spacy.load("ML-spaCy/output/model-best")

# Namespace-Map für TEI
ns = {'tei': 'http://www.tei-c.org/ns/1.0'}

# Entität-Typen zu Tags zuordnen
tag_map = {"PER": "persName", "LOC": "placeName", "DATE": "date"}

# Ordner mit XML-Dateien
input_dir = "data/5.3-TEI-Modellierung"

# XML-Parser konfigurieren
parser = ET.XMLParser(remove_blank_text=True)

# Alle XML-Dateien im Ordner rekursiv durchsuchen
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 4o

Gemeinsam nutzen

```
# Alle XML-Dateien im Verzeichnis rekursiv durchsuchen
for root_dir, _, files in os.walk(input_dir):
    for file_name in files:
        if file_name.endswith(".xml"):
            file_path = os.path.join(root_dir, file_name)
            print(f"Verarbeite: {file_path}")

            # XML-Datei laden
            tree = ET.parse(file_path, parser)
            root = tree.getroot()

            # Alle <p>-Elemente finden
            p_elems = root.xpath('//*[tei:text//tei:p]', namespaces=ns)

            # Jeden Absatz bearbeiten
            for p_elem in p_elems:
                # Leere Absätze überspringen
                if p_elem.text is None and len(p_elem) == 0:
                    continue

                # Text aus dem Absatz holen und bereinigen
                full_text = "".join(p_elem.itertext())
                cleaned_text = re.sub(r'(?<=\\d)\\.(?!=\\s{\\$})', '', full_text).strip()

                # Absatz leeren
                p_elem.clear()

                # In Sätze aufteilen (am Punkt + Leerzeichen)
                sentences = [s.strip() for s in cleaned_text.split('. ') if s.strip()]

                for sentence in sentences:
                    # ...
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 4o

Gemeinsam nutzen

```
for sentence in sentences:
    doc = ner_model(sentence)
    ents = [ent for ent in doc.ents if ent.label_ in tag_map]

    if not ents:
        p_elem.text = (p_elem.text or '') + sentence + ". "
        continue

    # Neues <s>-Element erstellen
    s_elem = ET.Element("s")
    last_idx = 0

    for ent in ents:
        # Text vor der Entität einfügen
        if ent.start_char > last_idx:
            chunk = sentence[last_idx:ent.start_char]
            if len(s_elem) == 0:
                s_elem.text = chunk
            else:
                s_elem[-1].tail = chunk

        # Entität als XML-Element einfügen
        tag = tag_map[ent.label_]
        ent_elem = ET.Element(tag)
        ent_elem.text = ent.text
        s_elem.append(ent_elem)

        last_idx = ent.end_char

    # Rest vom Satz + Punkt einfügen
    remaining = sentence[last_idx:] + ". "
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 4o

Gemeinsam nutzen



```
# Rest vom Satz + Punkt einfügen
remaining = sentence[last_idx:] + ". "
if len(s_elem) == 0:
    s_elem.text = remaining
else:
    s_elem[-1].tail = remaining

p_elem.append(s_elem)

# Datei überschreiben
tree.write(file_path, encoding='utf-8', pretty_print=True, xml_declaration=
print(f"Überschrieben: {file_path}")
```

✅ Was dieser neue Code tut:

- Läuft durch alle XML-Dateien in `data/5.3_TEI-Modellierung` (auch Unterordner).
- Wendet das spaCy-NER-Modell an.
- Fügt `<s>`-Satz-Elemente ein und ersetzt erkannte Entitäten mit den passenden TEI-Tags.
- Speichert die bearbeiteten Dateien direkt unter dem Originalpfad.

📌 Hinweis:

Falls du lieber Backups behalten möchtest, sag Bescheid – ich kann dir auch eine Version erstellen, die die Originaldateien sichert, bevor sie überschrieben werden.

Möchtest du das?

↓

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

5.4.1 NER (Erstellung von Trainingsdaten)

ChatGPT 5 ▾

↑ Gemeinsam nutzen ...



Ich möchte mithilfe von llama llama3 einen deutschen Satz erzeugen, in dem die folgende Person, der folgende Ort und das Datum vorkommen. Außerdem möchte ich ich möglichst viele der angegebenen Verben verwenden. Der Satz soll grammatikalisch korrekt und natürlich klingen.

Person: Maria Schneider
Ort: Berlin
Datum: im 12. Mai 1998
Verben: reisen, studieren, feiern, besuchen, arbeiten

Beispielausgabe: Ein sinnvoller deutscher Satz, in dem Maria Schneider, Berlin und das Datum im 12. Mai 1998 vorkommen.

Schreibe bitte für diese Aufgabe ein Python Code. Verwende bitte llama Bibliothek, das llama3-Model habe ich schon am meinen Computer installiert.

Nachgedacht für 14s

Hier ist ein kompaktes Python-Skript, das mit der `ollama`-Bibliothek (Model „llama3“) genau einen natürlichen, grammatikalisch korrekten deutschen Satz erzeugt. Der Satz enthält die vorgegebenen Elemente und versucht, möglichst viele der Verben sinnvoll zu verwenden. Falls die Datumspräposition unpassend ist (wie „im 12. Mai 1998“), wird sie automatisch zu „am 12. Mai 1998“ korrigiert.

```
python
# requirements: pip install ollama
# Hinweis: Das llama3-Modell ist laut deiner Angabe bereits lokal installiert.
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#)

ChatGPT 5 ▾

↑ Gemeinsam nutzen ...



```
python
# requirements: pip install ollama
# Hinweis: Das llama3-Modell ist laut deiner Angabe bereits lokal installiert.

import re
import ollama

MODEL = "llama3"

# Vorgaben
person = "Maria Schneider"
ort = "Berlin"
datum_roh = "im 12. Mai 1998" # wird unten ggf. zu 'am 12. Mai 1998' korrigiert
verben = ["reisen", "studieren", "feiern", "besuchen", "arbeiten"]

def normalisiere_datum(datum: str) -> str:
    """
    Korrigiert unübliche Präpositionen für Tagesdaten (z. B. 'im 12. Mai 1998' -> 'am 12. Mai 1998').
    """
    # Wenn am Anfang "im " steht und danach eine Tageszahl kommt, ersetze durch "am ".
    if datum.strip().lower().startswith("im "):
        # grobe Heuristik: enthält eine Ziffer (Tag) und einen Monatsnamen
        if re.search(r"\b\d{1,2}\.\s*(Januar|Februar|März|April|Mai|Juni|Juli|August|September|Oktober|November|Dezember)", datum):
            return re.sub(r"^im\s*", "am ", datum, flags=re.IGNORECASE)
    return datum

datum = normalisiere_datum(datum_roh)

SYSTEM_PROMPT = (
    "Du bist ein präziser deutscher Texter. "
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#)



ChatGPT 5 ▾

[Gemeinsam nutzen](#) ...

```

resp = ollama.chat(
    model=MODEL,
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt},
    ],
    options={
        "temperature": TEMPERATURE,
        "top_p": 0.9,
        "num_predict": 128,
        **({"seed": SEED} if SEED is not None else {}),
    },
)
text = resp["message"]["content"].strip()
# Minimal säubern
text = text.replace(" ", "").replace(" ", "").replace("'", "").strip()
# Falls das Modell mehrere Sätze geliefert hat, nehme nur bis zum ersten Punkt.
if text.count(".") >= 1:
    first = text.split(".", 1)[0].strip() + "."
    text = first
# Sicherstellen Punkt am Ende
if not text.endswith("."):
    text += "."
return text

# === HAUPTLAUF =====

def main():
    if SEED is not None:
        random.seed(SEED)

```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

[Gemeinsam nutzen](#) ...

```

spans = []
for surface, label in entities:
    start = text.find(surface)
    if start == -1:
        raise ValueError(f"Entität nicht im Text gefunden: {surface!r} ({label})")
    end = start + len(surface)
    spans.append((start, end, label))
# Spans nach Start sortieren (optional, hilft beim Debuggen)
spans.sort(key=lambda t: t[0])
return spans

def erzeuge_satz_mit_ollama(person: str, loc: str, date_str: str, verben: List[str]) -> str:
    """
    Fragt das Modell nach GENAU EINEM Satz, der person, loc und date_str
    WORTWÖRTLICH enthält, und möglichst viele Verben sinnvoll konjugiert.
    """
    system_prompt = (
        "Du bist ein präziser deutscher Texter.\n"
        "Erzeuge GENAU EINEN natürlichen, grammatikalisch korrekten deutschen Satz.\n"
        "WICHTIG:\n"
        f"- Verwende die folgenden Ausdrücke WORTWÖRTLICH im Satz: {person} ; {loc} ; {date_str} ;\n"
        f"- Versuche, möglichst viele dieser Verben natürlich zu integrieren (konjugiert).\n"
        "- Kein weiterer Text, keine Anführungszeichen, keine Erklärungen.\n"
        "- Genau EIN Satz, endend mit einem Punkt."
    )

    user_prompt = "Gib nur den einen Satz aus (ohne Anführungszeichen)."
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

↑ Gemeinsam nutzen ...

```
def normalisiere_monat(s: str) -> str:
    """
    Normalisiert Monatsnamen (erster Buchstabe groß, Rest klein; deutsche Namen erlaubt)
    """
    s = s.strip().strip(".")
    # Ersten Buchstaben groß, Rest klein – aber Umlaute/B bleiben erhalten
    return s[:1].upper() + s[1:].lower() if s else s

def baue_datum_tag_monat_jahr(day: int, month: str, year: int) -> str:
    """
    Baut ein deutsches Datum mit sinnvoller Präposition:
    'am {day}. {month} {year}'
    """
    month = normalisiere_monat(month)
    return f"am {day}. {month} {year}"

def wähle_verben(alle_verben: List[str], min_n=3, max_n=5) -> List[str]:
    n = min(len(alle_verben), random.randint(min_n, max_n))
    return random.sample(alle_verben, n) if alle_verben else []

def enthaelt_alle(satz: str, person: str, loc: str, date_str: str) -> bool:
    return (person in satz) and (loc in satz) and (date_str in satz)

def berechne_spans(text: str, entities: List[Tuple[str, str]]) -> List[Tuple[int, int, str]]:
    """
    entities: Liste von (exakter_text, label)
    Gibt Spans als (start, end, label) zurück. Nimmt jeweils das erste Vorkommen.
    """
    spans = []
    for surface, label in entities:
        start = text.find(surface)
        if start == -1:
            continue
        end = start + len(surface)
        spans.append((start, end, label))
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

↑ Gemeinsam nutzen ...

```
def lade_zeilen(pfad: str) -> List[str]:
    with open(pfad, "r", encoding="utf-8") as f:
        lines = [re.sub(r"\s+", " ", ln.strip()) for ln in f.readlines()]
    # Filtere leere Zeilen
    return [ln for ln in lines if ln]

def parse_day(s: str) -> int:
    """
    Extrahiert den Tag als int aus einer Zeile (z.B. '12', '12.', '12 ') -> 12
    """
    m = re.search(r"\d{1,2}", s)
    if not m:
        raise ValueError(f"Kein numerischer Tag in: {s}")
    d = int(m.group(0))
    if not (1 <= d <= 31):
        raise ValueError(f"Ungültiger Tag: {d}")
    return d

def parse_year(s: str) -> int:
    """
    Extrahiert das Jahr als int (z.B. '1998', '2021 ') -> 1998/2021
    """
    m = re.search(r"\d{3,4}", s)
    if not m:
        raise ValueError(f"Kein Jahr in: {s}")
    y = int(m.group(0))
    return y

def normalisiere_monat(s: str) -> str:
    """
    Normalisiert Monatsnamen (erster Buchstabe groß, Rest klein; deutsche Namen erlaubt)
    """
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[Gemeinsam nutzen](#) ...


```
def finde_dateien(daten_ordner: str, keywords: Dict[str, List[str]])
    """
    Durchsucht rekursiv daten_ordner nach .txt-Dateien und ordnet sie anhand
    einfacher Namens-Heuristiken Kategorien zu.
    Nimmt die erste passende Datei pro Kategorie.
    """
    zuordnung = {k: None for k in keywords.keys()}

    for root, _, files in os.walk(daten_ordner):
        for fn in files:
            if not fn.lower().endswith(".txt"):
                continue
            lower = fn.lower()
            for k, patterns in keywords.items():
                if zuordnung[k] is not None:
                    continue
                if any(p in lower for p in patterns):
                    zuordnung[k] = os.path.join(root, fn)

    fehlend = [k for k, v in zuordnung.items() if v is None]
    if fehlend:
        raise FileNotFoundError(
            "Folgende Kategorien konnten nicht über Dateinamen-Heuristiken zugeordnet w
            + ", ".join(fehlend)
            + f"\nBitte benenne die Dateien sprechend (z.B. 'personen.txt', 'tage.txt',
            )
    return zuordnung

def lade_zeilen(pfad: str) -> List[str]:
    with open(pfad, "r", encoding="utf-8") as f:
        lines = [re.sub(r"\s+", " ", ln.strip()) for ln in f.readlines()]
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

[Gemeinsam nutzen](#) ...


```
import os
import re
import random
from typing import List, Tuple, Dict
import ollama

MODEL = "llama3"

# === KONFIGURATION ===
DATEN_ORDNER = "./daten" # <-- Pfad zu deinem Ordner mit den .txt-Dateien
ANZAHL_SAETZE = 10 # wie viele Beispiele erzeugt werden sollen
MAX_RETRIES_PER_SAMPLE = 2 # wie oft nachgebessert wird, wenn etwas fehlt
TEMPERATURE = 0.6
SEED = 42 # für reproduzierbare Zufälligkeit (None = variabel)

# Falls deine Dateien anders benannt sind, passe die Schlagwörter unten an.
DATEI_KEYWORDS = {
    "persons": ["person", "personen", "names", "namen"],
    "days": ["tag", "day", "tage", "days"],
    "months": ["monat", "monate", "month", "months"],
    "years": ["jahr", "jahre", "year", "years"],
    "locations": ["ort", "orte", "location", "locations", "stadt", "staedte", "städte",
    "verbs": ["verb", "verben", "verbs"],
}

# === HILFSFUNKTIONEN ===

def finde_dateien(daten_ordner: str, keywords: Dict[str, List[str]]) -> Dict[str, str]:
    """
    Durchsucht rekursiv daten_ordner nach .txt-Dateien und ordnet sie anhand
    einfacher Namens-Heuristiken Kategorien zu.
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[Gemeinsam nutzen](#) ...


danke es hat funktioniert. Nun möchte ich, dass es mehrere Sätzen generiert werden und sie müssen folgende Format haben:

```
[
  ("Tokyo Tower is 333m tall.", [(0, 11, "BUILDING")]),
]
```

Anmontiere bitte den Text folgende: Personennamen - PERSON; Datum - DATE; Ortsname LOC

Die Personennamen, Tag, Monatsname, Jahr, Ortsnamen und Verben habe ich in separate txt-Dateien gespeichert. Der Code soll Datum random zusammenfügen. Um den Ordner durchzusuchen, verwende bitte os-Bibliothek

Nachgedacht für 15s

Hier ist ein robustes Python-Skript, das mit `ollama` (llama3) **mehrere** deutsche Sätze erzeugt und sie im gewünschten Format inkl. **Span-Annotationen** für `PERSON`, `DATE` und `LOC` ausgibt.

Es durchsucht mit `os` einen Datenordner nach den benötigten TXT-Dateien (Personen, Tage, Monate, Jahre, Orte, Verben), setzt das Datum zufällig zusammen, bittet das Modell pro Beispiel um **genau einen** Satz (ohne Anführungszeichen) und validiert, dass die drei Pflicht-Entities **wortwörtlich** im Satz vorkommen. Die Spans werden als `(start, end, label)` berechnet.

```
python
# requirements:
# pip install ollama
# Hinweise:
# - Das llama3-Modell ist laut deiner A... bereits lokal installiert.
# - Lege deine TXT-Dateien in einem Verzeichnis ab, z.B. ./daten
# (main.py, Katastrophen, Personen, Tage, Monate, Jahre, Orte, Verben)

+ Stelle irgendeine Frage
```

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[Gemeinsam nutzen](#) ...


```
reprompt = SYSTEM_PROMPT + (
    f"\n\nDas vorherige Ergebnis fehlte bei: {', '.join(fehlegendes)} if fehlegendes
    "\nFormuliere neu und liefere GENAU EINEN Satz."
    + zusatz
)
satz = chat_request(reprompt)
retries += 1

# Falls das Modell einen Punkt zu viel/zu wenig setzt, minimal bereinigen.
satz = satz.replace(" ", "").replace("(", "").replace(")", "").strip()
# Sicherstellen: endet mit Punkt.
if not satz.endswith("."):
    satz += "."

return satz

if __name__ == "__main__":
    satz = generiere_satz()
    print(satz)
```

Was das Skript macht:

- Korrigiert das Datum zu einer natürlichen Form („am 12. Mai 1998“).
- Baut einen strengen System-Prompt, der genau **einen** Satz verlangt, der **Person, Ort und Datum** enthält und möglichst viele der Verben **konjugiert** integriert.
- Prüft, ob alle Pflichtangaben tatsächlich vorkommen und versucht **einmal** nachzubessern, falls etwas fehlt.
- Druckt am Ende nur den generierten Satz.



+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[Gemeinsam nutzen](#) ...


```

SYSTEM_PROMPT = (
    "Du bist ein präziser deutscher Texter. "
    "Erzeuge GENAU EINEN grammatikalisch korrekten und natürlichen deutschen Satz.\n\n"
    "Muss enthalten:\n"
    f"* Person: {person}\n"
    f"* Ort: {ort}\n"
    f"* Datum (mit passender Präposition): {datum}\n"
    "Verwende möglichst viele der folgenden Verben in sinnvoll konjugierter Form, ohne "
    f"{'', '.join(verben)}\n\n"
    "Weitere Anforderungen:\n"
    "- Genau ein Satz (ein abschließender Punkt). Keine Anführungszeichen, keine Erklär "
    "- Korrekte Groß-/Kleinschreibung, natürliches Deutsch."
)

USER_PROMPT = (
    "Gib nur den Satz aus, ohne Anführungszeichen und ohne zusätzliche Kommentare."
)

def enthaelt_alle_plichtangaben(text: str) -> bool:
    # Pflicht: exakte Namens- und Ortsnennung; Datum als normalisierte Zeichenkette
    return all(kw in text for kw in [person, ort, datum])

def generiere_satz(max_retries: int = 1, temperature: float = 0.6, seed: int | None = 4
    """
    Fragt das LLM an und validiert, dass Person, Ort und Datum enthalten sind.
    Versucht bei Fehlschlag einmal nachzubessern.
    """
    def chat_request(sys_prompt: str):
        return ollama.chat(
            model=MODEL,
            messages=[
                sys_prompt,
                USER_PROMPT,
            ],
            options={
                "temperature": temperature,
                "top_p": 0.9,
                "num_predict": 128,
                **({"seed": seed} if seed is not None else {}),
            },
        )["message"][0]["content"].strip()

    satz = chat_request(SYSTEM_PROMPT)

    # Validierung und ggf. Nachbesserung
    retries = 0
    while (not enthaelt_alle_plichtangaben(satz) or satz.count(".") != 1) and retries < 3:
        fehlendes = []
        if person not in satz: fehlendes.append("Person")
        if ort not in satz: fehlendes.append("Ort")
        if datum not in satz: fehlendes.append("Datum")
        zusatz = (
            "\n\nAchte zusätzlich darauf, möglichst viele der Verben zu verwenden: "
            + "{'', '.join(verben)}."
        )
        reprompt = SYSTEM_PROMPT + (
            f"\n\nDas vorherige Ergebnis war bei {'', '.join(fehlendes)}\n\n"
            f"Formuliere neu und liefere GENAU EINEN Satz."
        )
        satz = chat_request(reprompt)
        retries += 1

    return satz

```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

[Gemeinsam nutzen](#) ...


```

def chat_request(sys_prompt: str):
    return ollama.chat(
        model=MODEL,
        messages=[
            {"role": "system", "content": sys_prompt},
            {"role": "user", "content": USER_PROMPT},
        ],
        options={
            "temperature": temperature,
            "top_p": 0.9,
            "num_predict": 128,
            **({"seed": seed} if seed is not None else {}),
        },
    )["message"][0]["content"].strip()

satz = chat_request(SYSTEM_PROMPT)

# Validierung und ggf. Nachbesserung
retries = 0
while (not enthaelt_alle_plichtangaben(satz) or satz.count(".") != 1) and retries < 3:
    fehlendes = []
    if person not in satz: fehlendes.append("Person")
    if ort not in satz: fehlendes.append("Ort")
    if datum not in satz: fehlendes.append("Datum")
    zusatz = (
        "\n\nAchte zusätzlich darauf, möglichst viele der Verben zu verwenden: "
        + "{'', '.join(verben)}."
    )
    reprompt = SYSTEM_PROMPT + (
        f"\n\nDas vorherige Ergebnis war bei {'', '.join(fehlendes)}\n\n"
        f"Formuliere neu und liefere GENAU EINEN Satz."
    )
    satz = chat_request(reprompt)
    retries += 1

return satz

```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...

```

reprompt = SYSTEM_PROMPT + (
    f"\n\nDas vorherige Ergebnis fehlte bei: {', '.join(gehlendes) if fehlendes
    "\nFormuliere neu und liefere GENAU EINEN Satz."
    + zusatz
)
satz = chat_request(reprompt)
retries += 1

# Falls das Modell einen Punkt zu viel/zu wenig setzt, minimal bereinigen.
satz = satz.replace(" ", "").replace(" ", "").replace("'", "").strip()
# Sicherstellen: endet mit Punkt.
if not satz.endswith("."):
    satz += "."

return satz

if __name__ == "__main__":
    satz = generiere_satz()
    print(satz)

```

Was das Skript macht:

- Korrigiert das Datum zu einer natürlichen Form („am 12. Mai 1998“).
- Baut einen strengen System-Prompt, der genau **einen** Satz verlangt, der **Person, Ort und Datum** enthält und möglichst viele der Verben **konjugiert** integriert.
- Prüft, ob alle Pflichtangaben tatsächlich vorkommen und versucht **einmal** nachzubessern, falls etwas fehlt.
- Druckt am Ende nur den generierten Satz.



+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...

danke es hat funktioniert. Nun möchte ich, dass es mehrere Sätzen generiert werden und sie müssen folgende Format haben:

```
[
    ("Tokyo Tower is 333m tall.", [(0, 11, "BUILDING")]),
]
```

Annotiere bitte den Text folgende: Personennamen - PERSON;
Datum - DATE; Ortsname LOC

Die Personennamen, Tag, Monatsname, Jahr, Ortsnamen und Verben habe ich in separate txt-Dateien gespeichert. Der Code soll Datum random zusammenfügen. Um den Ordner durchzusuchen, verwende bitte os-Bibliothek

Nachgedacht für 15s

Hier ist ein robustes Python-Skript, das mit `ollama` (llama3) **mehrere** deutsche Sätze erzeugt und sie im gewünschten Format inkl. **Span-Annotationen** für `PERSON`, `DATE` und `LOC` ausgibt.

Es durchsucht mit `os` einen Datenordner nach den benötigten TXT-Dateien (Personen, Tage, Monate, Jahre, Orte, Verben), setzt das Datum zufällig zusammen, bittet das Modell pro Beispiel um **genau einen** Satz (ohne Anführungszeichen) und validiert, dass die drei Pflicht-Entities **wortwörtlich** im Satz vorkommen. Die Spans werden als `(start, end, label)` berechnet.

```

python

# requirements:
# pip install ollama
# Hinweise:
# - Das llama3-Modell ist laut deiner A... bereits lokal installiert.
# - Lege deine TXT-Dateien in einem Verzeichnis ab, z.B. ./daten
# (Name von Kategorie: Personen, Tage, Monate, Jahre, Orte, Verben)

```

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...

```
import os
import re
import random
from typing import List, Tuple, Dict
import ollama

MODEL = "llama3"

# === KONFIGURATION ===
DATEN_ORDNER = "./daten" # <-- Pfad zu deinem Ordner mit den .txt-Dateien
ANZAHL_SAETZE = 10 # wie viele Beispiele erzeugt werden sollen
MAX_RETRIES_PER_SAMPLE = 2 # wie oft nachgebessert wird, wenn etwas fehlt
TEMPERATURE = 0.6
SEED = 42 # für reproduzierbare Zufälligkeit (None = variabel)

# Falls deine Dateien anders benannt sind, passe die Schlagwörter unten an.
DATEI_KEYWORDS = {
    "persons": ["person", "personen", "names", "namen"],
    "days": ["tag", "day", "tage", "days"],
    "months": ["monat", "monate", "month", "months"],
    "years": ["jahr", "jahre", "year", "years"],
    "locations": ["ort", "orte", "location", "locations", "stadt", "staedte", "städte"],
    "verbs": ["verb", "verben", "verbs"],
}

# === HILFSFUNKTIONEN ===

def finde_dateien(daten_ordner: str, keywords: Dict[str, List[str]]) -> Dict[str, str]:
    """
    Durchsucht rekursiv daten_ordner nach .txt-Dateien und ordnet sie anhand
    einfacher Namens-Heuristiken Kategorien zu.
    """
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...

```
def finde_dateien(daten_ordner: str, keywords: Dict[str, List[str]]) -> Dict[str, str]:
    """
    Durchsucht rekursiv daten_ordner nach .txt-Dateien und ordnet sie anhand
    einfacher Namens-Heuristiken Kategorien zu.
    Nimmt die erste passende Datei pro Kategorie.
    """
    zuordnung = {k: None for k in keywords.keys()}

    for root, _, files in os.walk(daten_ordner):
        for fn in files:
            if not fn.lower().endswith(".txt"):
                continue
            lower = fn.lower()
            for k, patterns in keywords.items():
                if zuordnung[k] is not None:
                    continue
                if any(p in lower for p in patterns):
                    zuordnung[k] = os.path.join(root, fn)

    fehlend = [k for k, v in zuordnung.items() if v is None]
    if fehlend:
        raise FileNotFoundError(
            "Folgende Kategorien konnten nicht über Dateinamen-Heuristiken zugeordnet werden: "
            + ", ".join(fehlend)
            + "\nBitte benenne die Dateien sprechend (z.B. 'personen.txt', 'tage.txt', ...)"
        )
    return zuordnung

def lade_zeilen(pfad: str) -> List[str]:
    with open(pfad, "r", encoding="utf-8") as f:
        lines = [re.sub(r"\s+", " ", ln.strip()) for ln in f.readlines()]
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...

```
def lade_zeilen(pfad: str) -> List[str]:
    with open(pfad, "r", encoding="utf-8") as f:
        lines = [re.sub(r"\s+", " ", ln.strip()) for ln in f.readlines()]
    # Filtere leere Zeilen
    return [ln for ln in lines if ln]

def parse_day(s: str) -> int:
    """
    Extrahiert den Tag als int aus einer Zeile (z.B. '12', '12.', '12 ') -> 12
    """
    m = re.search(r"\d{1,2}", s)
    if not m:
        raise ValueError(f"Kein numerischer Tag in: {s}")
    d = int(m.group(0))
    if not (1 <= d <= 31):
        raise ValueError(f"Ungültiger Tag: {d}")
    return d

def parse_year(s: str) -> int:
    """
    Extrahiert das Jahr als int (z.B. '1998', '2021 ') -> 1998/2021
    """
    m = re.search(r"\d{3,4}", s)
    if not m:
        raise ValueError(f"Kein Jahr in: {s}")
    y = int(m.group(0))
    return y

def normalisiere_monat(s: str) -> str:
    """
    Normalisiert Monatsnamen (erster Buchstabe groß, Rest klein; deutsche Namen erlaubt)
    """
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...

```
def normalisiere_monat(s: str) -> str:
    """
    Normalisiert Monatsnamen (erster Buchstabe groß, Rest klein; deutsche Namen erlaubt)
    """
    s = s.strip().strip(".")
    # Ersten Buchstaben groß, Rest klein – aber Umlaute/ß bleiben erhalten
    return s[:1].upper() + s[1:].lower() if s else s

def baue_datum_tag_monat_jahr(day: int, month: str, year: int) -> str:
    """
    Baut ein deutsches Datum mit sinnvoller Präposition:
    'am {day}. {month} {year}'
    """
    month = normalisiere_monat(month)
    return f"am {day}. {month} {year}"

def wähle_verben(alle_verben: List[str], min_n=3, max_n=5) -> List[str]:
    n = min(len(alle_verben), random.randint(min_n, max_n))
    return random.sample(alle_verben, n) if alle_verben else []

def enthaelt_alle(satz: str, person: str, loc: str, date_str: str) -> bool:
    return (person in satz) and (loc in satz) and (date_str in satz)

def berechne_spans(text: str, entities: List[Tuple[str, str]]) -> List[Tuple[int, int]]:
    """
    entities: Liste von (exakter_text, label)
    Gibt Spans als (start, end, label) zurück. Nimmt jeweils das erste Vorkommen.
    """
    spans = []
    for surface, label in entities:
        start = text.find(surface)
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

[Gemeinsam nutzen](#) ...

```

spans = []
for surface, label in entities:
    start = text.find(surface)
    if start == -1:
        raise ValueError(f"Entität nicht im Text gefunden: {surface!r} ({label})")
    end = start + len(surface)
    spans.append((start, end, label))
# Spans nach Start sortieren (optional, hilft beim Debuggen)
spans.sort(key=lambda t: t[0])
return spans

def erzeuge_satz_ollama(person: str, loc: str, date_str: str, verben: List[str]) ->
"""
Fragt das Modell nach GENAU EINEM Satz, der person, loc und date_str
WORTWÖRTLICH enthält, und möglichst viele Verben sinnvoll konjugiert.
"""
system_prompt = (
    "Du bist ein präziser deutscher Texter.\n"
    "Erzeuge GENAU EINEN natürlichen, grammatikalisch korrekten deutschen Satz.\n"
    "WICHTIG:\n"
    f"- Verwende die folgenden Ausdrücke WORTWÖRTLICH im Satz: {person}; {loc}; {
    f"- Versuche, möglichst viele dieser Verben natürlich zu integrieren (konjugier
    "- Kein weiterer Text, keine Anführungszeichen, keine Erklärungen.\n"
    "- Genau EIN Satz, endend mit einem Punkt."
)

user_prompt = "Gib nur den einen Satz aus (ohne Anführungszeichen)."
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

[Gemeinsam nutzen](#) ...

```

resp = ollama.chat(
    model=MODEL,
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt},
    ],
    options={
        "temperature": TEMPERATURE,
        "top_p": 0.9,
        "num_predict": 128,
        **({"seed": SEED} if SEED is not None else {}),
    },
)
text = resp["message"]["content"].strip()
# Minimal säubern
text = text.replace(" ", "").replace(" ", "").replace("'", "").strip()
# Falls das Modell mehrere Sätze geliefert hat, nehme nur bis zum ersten Punkt.
if text.count(".") >= 1:
    first = text.split(".", 1)[0].strip() + "."
    text = first
# Sicherstellen Punkt am Ende
if not text.endswith("."):
    text += "."
return text

# == HAUPTLAUF ==

def main():
    if SEED is not None:
        random.seed(SEED)
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

↑ Gemeinsam nutzen ...

```

dateipfade = finde_dateien(DATEN_ORDNER, DATEI_KEYWORDS)

personen = lade_zeilen(dateipfade["persons"])
tage_raw = lade_zeilen(dateipfade["days"])
monate = lade_zeilen(dateipfade["months"])
jahre_raw = lade_zeilen(dateipfade["years"])
orte = lade_zeilen(dateipfade["locations"])
verben_alle = lade_zeilen(dateipfade["verbs"])

# Parsen
tage = [parse_day(x) for x in tage_raw]
jahre = [parse_year(x) for x in jahre_raw]
monate = [normalisiere_monat(m) for m in monate]

ausgaben: List[Tuple[str, List[Tuple[int, int, str]]]] = []

for _ in range(ANZAHL_SATZES):
    person = random.choice(personen)
    ort = random.choice(orte)
    tag = random.choice(tage)
    monat = random.choice(monate)
    jahr = random.choice(jahre)
    date_str = baue_datum_tag_monat_jahr(tag, monat, jahr)
    verben = wähle_verben(verben_alle)

# Generieren + Validieren
satz = erzeuge_satz_mit_ollama(person, ort, date_str, verben)

retries = 0
while (not enthaelt_alle(satz, person, ort, date_str) or satz.count(".") != 1):
    # Nachbesserungs-Prompt mit Hinweis, was fehlte
    ...

```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

↑ Gemeinsam nutzen ...

```

retries = 0
while (not enthaelt_alle(satz, person, ort, date_str) or satz.count(".") != 1):
    # Nachbesserungs-Prompt mit Hinweis, was fehlte
    fehlendes = []
    if person not in satz: fehlendes.append("PERSON")
    if ort not in satz: fehlendes.append("LOC")
    if date_str not in satz: fehlendes.append("DATE")

    reprompt_system = (
        "Erzeuge GENAU EINEN deutschen Satz, endend mit einem Punkt.\n"
        f"Die folgenden Ausdrücke MÜSSEN WORTWÖRTLICH enthalten sein: {person}\n"
        f"Verwende möglichst viele dieser Verben: {', '.join(verben) if verben\n"
        f"Vorher fehlend: {', '.join(gehlendes) if fehlendes else 'Format'}.\n"
        "Kein weiterer Text, keine Anführungszeichen."
    )

    resp = ollama.chat(
        model=MODEL,
        messages=[
            {"role": "system", "content": reprompt_system},
            {"role": "user", "content": "Nur den einen Satz ausgeben."},
        ],
        options={
            "temperature": TEMPERATURE,
            "top_p": 0.9,
            "num_predict": 128,
            **({"seed": SEED} if SEED is not None else {}),
        },
    )
    satz = resp["message"]["content"].strip().replace(" ", "").replace(" ", "")
    # Nur den ersten Satzteil bis zum ersten Punkt
    if "." in satz:
        satz = satz.split(".")[-1].strip()

```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

Gemeinsam nutzen ...



```

if not satz.endswith("."):
    satz += "."
    retries += 1

# Falls trotz Nachbesserung etwas fehlt, setze zur Not eine einfache Schablone
# damit die Ausgabe immer korrekt annotierbar ist.
if not enthaelt_alle(satz, person, ort, date_str):
    # einfache, natürliche Vorlage inkl. einiger Verben (falls vorhanden)
    verba = ", ".join(verben[:2]) if verben else "studierte"
    satz = f"{person} war {date_str} in {ort} und konnte {verba}; anschließend
if not satz.endswith("."):
    satz += "."

# Spans berechnen
spans = berechne_spans(
    satz,
    [
        (person, "PERSON"),
        (date_str, "DATE"),
        (ort, "LOC"),
    ],
)
ausgaben.append((satz, spans))

# Ausgabe im gewünschten Python-Listenformat
# Beispiel:
# [
#   ("Satz 1.", [(start, end, "PERSON"), (start, end, "DATE"), (start, end, "LOC")])
#   ...
# ]
print("")
for i, (text, spans) in enumerate(ausgaben):

```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...



```

for i, (text, spans) in enumerate(ausgaben):
    # Escape für eingebettete Anführungszeichen etc.
    escaped = text.replace("\\", "\\\").replace("'", "\\'")
    # Wir geben String mit doppelten Anführungszeichen aus (wie im Beispiel).
    spans_str = ", ".join(f"({s}, {e}, \"{lbl}\")" for (s, e, lbl) in spans)
    comma = ", " if i < len(ausgaben) - 1 else ""
    print(f' ({escaped}), [{spans_str}]{comma}')
print("")

if __name__ == "__main__":
    main()

```

Was du anpassen kannst:

- `DATEN_ORDNER` auf den Pfad zu deinem Datenordner.
- `ANZAHL_SAETZE` für die gewünschte Anzahl.
- Benennungen deiner TXT-Dateien: Wenn nötig, erweitere `DATEI_KEYWORDS`.

Erwartetes Ausgabeformat (Beispielstruktur):

```

python

[
  ("Maria Schneider reiste am 12. Mai 1998 nach Berlin, besuchte Freunde, feierte ihren
  ...
  ]

```

Damit bekommst du eine Liste von `(Satz, Spans)`-Tupeln, wobei die Spans die Positionen der drei Entitäten `PERSON`, `DATE`, `LOC` im Satz angeben.



+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...

kannst du bitte das Prompt ändern: (1.) Erstelle einen deutschen Satz über die Person die sich in Ort aufhält oder dort etwas tut. Verwende exakt diesen Namen und exakt diesen Ort. Nutze genau eines der folgenden Verben. Der Satz muss das Datum enthalten, allerdings nicht am Satzanfang – es soll möglichst in der Satzmitte oder am Ende erscheinen. Gib ausschließlich den vollständigen Satz aus – keine Erklärungen, Kommentare oder Einleitungen.

Alles klar 🙌 – ich habe dein gewünschtes Prompt in das Skript eingebaut.

Das Modell wird nun so angewiesen, dass es **genau einen Satz** erzeugt, in dem:

- die Person **exakt** so wie vorgegeben vorkommt,
- der Ort **exakt** so wie vorgegeben vorkommt,
- **genau ein Verb** aus der Verbliste benutzt wird,
- das Datum enthalten ist, **aber nicht am Satzanfang** (besser Mitte/Ende),
- und **wirklich nur der Satz** ausgegeben wird (ohne Kommentare).

Hier der angepasste Teil des Codes (ich habe nur die Prompt-Generierung geändert):

```
python
def erzeuge_satz_mit_ollama(person: str, loc: str, date_str: str, verben: List[str]) -> str:
    """
    Fragt das Modell nach GENAU EINEM Satz:
    - Person und Ort exakt enthalten
    - Datum enthalten, aber nicht am Satzanfang
    - genau EIN Verb aus der Liste verwenden
    """
    # Wähle genau ein Verb
    verb = random.choice(verben) if verben else "erhalten"
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...

```
system_prompt = (
    "Erstelle einen deutschen Satz über die Person, die sich im angegebenen Ort auf"
    f"- Verwende exakt diesen Namen: {person}\n"
    f"- Verwende exakt diesen Ort: {loc}\n"
    f"- Nutze GENAU dieses Verb: {verb}\n"
    f"- Der Satz MUSS das Datum {date_str} enthalten, jedoch NICHT am Satzanfang (s"
    "- Gib ausschließlich den vollständigen Satz aus – keine Erklärungen, Kommentar"
    "- Genau EIN Satz, endend mit einem Punkt."
)

user_prompt = "Nur den vollständigen Satz ausgeben."

resp = ollama.chat(
    model=MODEL,
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt},
    ],
    options={
        "temperature": TEMPERATURE,
        "top_p": 0.9,
        "num_predict": 128,
        **({"seed": SEED} if SEED is not None else {}),
    },
)

text = resp["message"]["content"].strip()
# Minimal säubern
text = text.replace(" ", "").replace("'''", "").replace("'", "").strip()
# Nur bis zum ersten Punkt (falls das Modell mehr liefert)
if "." in text:
    text = text.split(".", 1)[0].strip() + "."
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...

```

if "." in text:
    text = text.split(".", 1)[0].strip() + "."
if not text.endswith("."):
    text += "."
return text

```

Kopieren Bearbeiten

Damit wird jetzt bei jedem Satz nur ein Verb genutzt.

☞ Soll ich den Rest des Skripts (z. B. `main()`) auch gleich auf ein Verb pro Satz umstellen, oder willst du mehrere Verben weiter als Auswahl mitgeben, aber jeweils nur eins verwenden lassen?

📄 📄 📄 📄 📄 📄 📄 📄

ich habe ein paar Änderungen gemacht, kannst du bitte die folgende Funktion integrieren:

```

def load_list_from_file(filename):
    with open(filename, 'r', encoding='utf-8') as file:
        return [line.strip() for line in file if line.strip()]

def generate_date(raw=False):
    tag = random.choice(datum_list)
    monat = random.choice(monat_list)
    jahr = random.choice(jahr_list)
    return (tag, monat, jahr) if raw else f"im {tag} {monat} {jahr}"

def find_span(text, target):
    start = text.find(target)
    return (start, start + len(target)) if start != -1 else None

```

Hier sind die Pfade zu txt-Dateien:

+ Stelle irgendeine Frage

🔊 🔊

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...

```

def find_span(text, target):
    start = text.find(target)
    return (start, start + len(target)) if start != -1 else None

```

Hier sind die Pfade zu txt-Dateien:
Daten laden

```

person_list_1 = load_list_from_file('input/person_list.txt')
person_list_2 = load_list_from_file('input/nachname_list.txt') # neu eingebunden
ortsname_list_1 = load_list_from_file('input/ortsname_list.txt')
ortsname_list_2 = load_list_from_file('input/ortsname_SA_E_list.txt') # neu eingebunden
jahr_list = load_list_from_file("input/jahr_list.txt")
monat_list = load_list_from_file("input/monat_list.txt")
datum_list = load_list_from_file("input/datum_list.txt")
verb_list = load_list_from_file("input/verb_list.txt")

```

Alles klar 🙌 – ich habe deine neuen Funktionen und die Dateipfade integriert.

Ich habe außerdem die bestehende Logik so angepasst, dass:

- `load_list_from_file()` für alle Listen genutzt wird.
- `generate_date()` ein zufälliges Datum erzeugt (mit Option `raw=True`, falls du die Teile einzeln brauchst).
- `find_span()` beim Annotieren der Entities verwendet wird.
- Personen- und Ortsnamen jetzt jeweils aus zwei Listen zusammengesetzt werden (z. B. Vorname + Nachname, oder Stadt + Zusatz).

Hier der integrierte Codeausschnitt (mit Fokus auf Änderungen):

python

Kopieren Bearbeiten

+ Stelle irgendeine Frage

🔊 🔊

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[Gemeinsam nutzen](#) ...


```
python
import random
import ollama

MODEL = "llama3"
TEMPERATURE = 0.6
SEED = 42

# -----
# Neue Hilfsfunktionen
# -----

def load_list_from_file(filename):
    with open(filename, 'r', encoding='utf-8') as file:
        return [line.strip() for line in file if line.strip()]

def generate_date(raw=False):
    tag = random.choice(datum_list)
    monat = random.choice(monat_list)
    jahr = random.choice(jahr_list)
    return (tag, monat, jahr) if raw else f"im {tag} {monat} {jahr}"

def find_span(text, target):
    start = text.find(target)
    return (start, start + len(target)) if start != -1 else None

# -----
# Daten laden
# -----
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[Gemeinsam nutzen](#) ...


```
person_list_1 = load_list_from_file('input/person_list.txt')
person_list_2 = load_list_from_file('input/nachname_list.txt') # neu
ortsname_list_1 = load_list_from_file('input/ortsname_list.txt')
ortsname_list_2 = load_list_from_file('input/ortsname_SA_E_list.txt') # neu
jahr_list = load_list_from_file("input/jahr_list.txt")
monat_list = load_list_from_file("input/monat_list.txt")
datum_list = load_list_from_file("input/datum_list.txt")
verb_list = load_list_from_file("input/verb_list.txt")

# -----
# Satzgenerierung
# -----

def erzeuge_satz_mit_ollama(person: str, loc: str, date_str: str, verb: str) -> str:
    system_prompt = (
        "Erstelle einen deutschen Satz über die Person, die sich im angegebenen Ort auf"
        f"- Verwende exakt diesen Namen: {person}\n"
        f"- Verwende exakt diesen Ort: {loc}\n"
        f"- Nutze GENAU dieses Verb: {verb}\n"
        f"- Der Satz MUSS das Datum {date_str} enthalten, jedoch NICHT am Satzanfang (s"
        "- Gib ausschließlich den vollständigen Satz aus - keine Erklärungen, Kommentar"
        "- Genau EIN Satz, endend mit einem Punkt."
    )

    resp = ollama.chat(
        model=MODEL,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": "Nur den vollständigen Satz ausgeben."},
        ],
        options={
            "temperature": TEMPERATURE,
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

[Gemeinsam nutzen](#) ...

```

options={
    "temperature": TEMPERATURE,
    "top_p": 0.9,
    "num_predict": 128,
    **({"seed": SEED} if SEED is not None else {}),
},
)
text = resp["message"]["content"].strip()
# Säubern
text = text.replace(" ", "").replace(" ", "").replace("'", "").strip()
if "." in text:
    text = text.split(".", 1)[0].strip() + "."
if not text.endswith("."):
    text += "."
return text

# -----
# Beispiel: mehrere Sätze generieren + Annotation
# -----

def main(anzahl=5):
    samples = []
    for _ in range(anzahl):
        # Person: Vor- + Nachname
        person = f"{random.choice(person_list_1)} {random.choice(person_list_2)}"
        # Ort: Hauptliste + optionaler Zusatz
        ort = f"{random.choice(ortsname_list_1)} {random.choice(ortsname_list_2)}"
        # Datum
        date_str = generate_date()
        # Verb: genau eins
        verb = random.choice(verb_list)

```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

[Gemeinsam nutzen](#) ...

```

satz = erzeuge_satz_mit_ollama(person, ort, date_str, verb)
# Spans suchen
spans = []
for surface, label in [(person, "PERSON"), (ort, "LOC"), (date_str, "DATE")]:
    span = find_span(satz, surface)
    if span:
        spans.append((span[0], span[1], label))

samples.append((satz, spans))

# Ausgabe im gewünschten Format
print("")
for i, (text, spans) in enumerate(samples):
    spans_str = ", ".join(f"({s}, {e}, \"{lbl}\")" for (s, e, lbl) in spans)
    comma = "," if i < len(samples)-1 else ""
    print(f'({text}, [{spans_str}]){comma}')
print("")

if __name__ == "__main__":
    main(anzahl=10)

```

Damit passiert Folgendes:

- `person` wird aus `person_list_1` (z. B. Vornamen) und `person_list_2` (Nachnamen) zusammengesetzt.
- `ort` wird aus `ortsname_list_1` und `ortsname_list_2` zusammengesetzt.
- `generate_date()` liefert das Datum im Format `{Tag} {Monat} {Jahr}`.
- Es wird ein zufälliges Verb gewählt.
- Jeder Satz wird mit `find_span()` annotiert: `(start, end, "PERSON"), (... , "LOC"), (... , "DATE")`.

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

[↑ Gemeinsam nutzen](#) ...

👉 Soll ich den Code auch so umbauen, dass bei `generate_date()` statt `"im {tag} {monat} {jahr}"` automatisch `"am {tag}. {monat} {jahr}"` herauskommt (also die natürlichere deutsche Form)?

📄 👍 🗨️ 🔊 📄 📄 📄

`ollama.chat()` funktioniert nicht so richtig, kannst du bitte im Code `ollama.generate()` verwenden?

Sehr gute Idee 👍 – `ollama.generate()` ist für einfache Textausgaben oft robuster als `ollama.chat()`.

Ich habe deinen Code so umgestellt, dass `ollama.generate()` genutzt wird. Dabei gebe ich den Prompt direkt an `prompt=...` weiter.

Hier die aktualisierte Version (nur die relevanten Teile geändert):

```
python
import random
import ollama

MODEL = "llama3"
TEMPERATURE = 0.6
SEED = 42

# -----
# Hilfsfunktionen (wie du gegeben hast)
# -----
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

[↑ Gemeinsam nutzen](#) ...

```
def load_list_from_file(filename):
    with open(filename, 'r', encoding='utf-8') as file:
        return [line.strip() for line in file if line.strip()]

def generate_date(raw=False):
    tag = random.choice(datum_list)
    monat = random.choice(monat_list)
    jahr = random.choice(jahr_list)
    return (tag, monat, jahr) if raw else f"im {tag} {monat} {jahr}"

def find_span(text, target):
    start = text.find(target)
    return (start, start + len(target)) if start != -1 else None

# -----
# Daten laden
# -----

person_list_1 = load_list_from_file('input/person_list.txt')
person_list_2 = load_list_from_file('input/nachname_list.txt')
ortsname_list_1 = load_list_from_file('input/ortsname_list.txt')
ortsname_list_2 = load_list_from_file('input/ortsname_SA_E_list.txt')
jahr_list = load_list_from_file("input/jahr_list.txt")
monat_list = load_list_from_file("input/monat_list.txt")
datum_list = load_list_from_file("input/datum_list.txt")
verb_list = load_list_from_file("input/verb_list.txt")

# -----
# Satzgenerierung mit ollama.generate()
# -----
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).





ChatGPT 5

[Gemeinsam nutzen](#) ...


```
def erzeuge_satz_mit_ollama(person: str, loc: str, date_str: str, Kopieren Bearbeiten)
    prompt = (
        "Erstelle einen deutschen Satz über die Person, die sich im angegebenen Ort auf
        f"- Verwende exakt diesen Namen: {person}\n"
        f"- Verwende exakt diesen Ort: {loc}\n"
        f"- Nutze GENAU dieses Verb: {verb}\n"
        f"- Der Satz MUSS das Datum {date_str} enthalten, jedoch NICHT am Satzanfang (s
        "- Gib ausschließlich den vollständigen Satz aus - keine Erklärungen, Kommentar
        "- Genau EIN Satz, endend mit einem Punkt."
    )

    response = ollama.generate(
        model=MODEL,
        prompt=prompt,
        options={
            "temperature": TEMPERATURE,
            "top_p": 0.9,
            "num_predict": 128,
            **({"seed": SEED} if SEED is not None else {}),
        }
    )

    text = response["response"].strip()
    # Säubern
    text = text.replace(" ", "").replace(" ", "").replace("'", "").strip()
    if "." in text:
        text = text.split(".", 1)[0].strip() + "."
    if not text.endswith("."):
        text += "."
    return text
```

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[Gemeinsam nutzen](#) ...


```
# ----- Kopieren Bearbeiten
# Beispiel: mehrere Sätze generieren + Annotation
# -----

def main(anzahl=5):
    samples = []
    for _ in range(anzahl):
        person = f"{random.choice(person_list_1)} {random.choice(person_list_2)}"
        ort = f"{random.choice(ortsname_list_1)} {random.choice(ortsname_list_2)}"
        date_str = generate_date()
        verb = random.choice(verb_list)

        satz = erzeuge_satz_mit_ollama(person, ort, date_str, verb)

        spans = []
        for surface, label in [(person, "PERSON"), (ort, "LOC"), (date_str, "DATE")]:
            span = find_span(satz, surface)
            if span:
                spans.append((span[0], span[1], label))

        samples.append((satz, spans))

    # Ausgabe im gewünschten Format
    print("")
    for i, (text, spans) in enumerate(samples):
        spans_str = ", ".join(f"({s}, {e}, \"{lbl}\")" for (s, e, lbl) in spans)
        comma = ", " if i < len(samples)-1 else ""
        print(f'  "{text}", [{spans_str}]{comma}')
    print("")

if __name__ == "__main__":
```

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...

```
if __name__ == "__main__":
    main(anzahl=10)
```

Kopieren Bearbeiten

Hauptänderung:

- `ollama.chat(...)` → `ollama.generate(...)`
- Zugriff auf Ausgabe jetzt über `response["response"]`

Soll ich den Code zusätzlich so umbauen, dass das Datum automatisch im Format „am {tag}. {Monat} {jahr}“ ausgegeben wird (natürliches Deutsch), anstatt „im {tag} {Monat} {jahr}“?

📄 📌 🔍 ↻ ⌂

kannst du bitte Prompt ändern und vollständiger Code ausgeben:

```
f"Erstelle einen deutschen Satz über die Person '{name}', "
f"die sich in '{ort}' aufhält oder dort etwas tut. "
f"Verwende exakt diesen Namen: {name}, und exakt diesen Ort: {ort}. "
f"Nutze genau eines der folgenden Verben: {', '.join(verb_sample)}. "
f"Der Satz muss das Datum {datum_str} enthalten, allerdings nicht am Satzanfang – "
f"es soll möglichst in der Satzmitte oder am Ende erscheinen. "
f"Gib ausschließlich den vollständigen Satz aus – keine Erklärungen, Kommentare oder Einleitungen."
```

Alles klar ✅ – ich habe den Prompt so umgestellt, wie du ihn angegeben hast, und den **vollständigen lauffähigen Code** vorbereitet.

Der Code verwendet nun `ollama.generate()`, lädt deine TXT-Dateien, wählt zufällig Person +

+ Stelle irgendeine Frage

🔊 🔇

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...

Alles klar ✅ – ich habe den Prompt so umgestellt, wie du ihn angegeben hast, und den **vollständigen lauffähigen Code** vorbereitet.

Der Code verwendet nun `ollama.generate()`, lädt deine TXT-Dateien, wählt zufällig Person + Nachname, Ort + Zusatz, Datum und **genau ein Verb**, und gibt die Sätze im gewünschten Format mit Annotationen zurück.

Hier der komplette Code:

```
python
import random
import ollama

MODEL = "llama3"
TEMPERATURE = 0.6
SEED = 42

# -----
# Hilfsfunktionen
# -----

def load_list_from_file(filename):
    with open(filename, 'r', encoding='utf-8') as file:
        return [line.strip() for line in file if line.strip()]

def generate_date(raw=False):
    tag = random.choice(datum_list)
    monat = random.choice(monat_list)
    jahr = random.choice(jahr_list)
    return (tag, monat, jahr) if raw else f"im {tag} {monat} {jahr}"
```

Kopieren Bearbeiten

+ Stelle irgendeine Frage

🔊 🔇

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[Gemeinsam nutzen](#) ...


```
def find_span(text, target):
    start = text.find(target)
    return (start, start + len(target)) if start != -1 else None

# -----
# Daten laden
# -----

person_list_1 = load_list_from_file('input/person_list.txt')
person_list_2 = load_list_from_file('input/nachname_list.txt')
ortsname_list_1 = load_list_from_file('input/ortsname_list.txt')
ortsname_list_2 = load_list_from_file('input/ortsname_SA_E_list.txt')
jahr_list = load_list_from_file("input/jahr_list.txt")
monat_list = load_list_from_file("input/monat_list.txt")
datum_list = load_list_from_file("input/datum_list.txt")
verb_list = load_list_from_file("input/verb_list.txt")

# -----
# Satzgenerierung mit ollama.generate()
# -----

def erzeuge_satz_mit_ollama(name: str, ort: str, datum_str: str, verb_sample: list) ->
    prompt = (
        f"Erstelle einen deutschen Satz über die Person '{name}', "
        f"die sich in '{ort}' aufhält oder dort etwas tut. "
        f"Verwende exakt diesen Namen: {name}, und exakt diesen Ort: {ort}. "
        f"Nutze genau eines der folgenden Verben: {', '.join(verb_sample)}. "
        f"Der Satz muss das Datum {datum_str} enthalten, allerdings nicht am Satzanfang "
        f"es soll möglichst in der Satzmitte oder am Ende erscheinen. "
        f"Gib ausschließlich den vollständigen Satz aus – keine Erklärungen, Kommentare "
    )
```

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[Gemeinsam nutzen](#) ...


```
response = ollama.generate(
    model=MODEL,
    prompt=prompt,
    options={
        "temperature": TEMPERATURE,
        "top_p": 0.9,
        "num_predict": 128,
        **({"seed": SEED} if SEED is not None else {}),
    )

text = response["response"].strip()
# Säubern
text = text.replace(" ", "").replace(" ", "").replace("'", "").strip()
if "." in text:
    text = text.split(".", 1)[0].strip() + "."
if not text.endswith("."):
    text += "."
return text

# -----
# Mehrere Sätze generieren + Annotation
# -----

def main(anzahl=5):
    samples = []
    for _ in range(anzahl):
        # Person: Vor- + Nachname
        name = f"{random.choice(person_list_1)} {random.choice(person_list_2)}"
        # Ort: Hauptliste + Zusatz
        ort = f"{random.choice(ortsname_list_1)} {random.choice(ortsname_list_2)}"
```

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[Gemeinsam nutzen](#) ...


```

name = f"{random.choice(person_list_1)} {random.choice(per
# Ort: Hauptliste + Zusatz
ort = f"{random.choice(ortsname_list_1)} {random.choice(ortsname_list_2)}"
# Datum
datum_str = generate_date()
# Verb: genau ein Sample
verb_sample = [random.choice(verb_list)]

satz = erzeuge_satz_mit_ollama(name, ort, datum_str, verb_sample)

# Spans suchen
spans = []
for surface, label in [(name, "PERSON"), (ort, "LOC"), (datum_str, "DATE")]:
    span = find_span(satz, surface)
    if span:
        spans.append((span[0], span[1], label))

samples.append((satz, spans))

# Ausgabe im gewünschten Format
print("")
for i, (text, spans) in enumerate(samples):
    spans_str = ", ".join(f"({s}, {e}, \"{lbl}\")" for (s, e, lbl) in spans)
    comma = ", " if i < len(samples)-1 else ""
    print(f'  "{text}", [{spans_str}]{comma}')
print("")

if __name__ == "__main__":
    main(anzahl=10)

```



+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[Gemeinsam nutzen](#) ...


🔑 Änderungen:

- Dein Prompt 1:1 eingebaut.
- `verb_sample` ist eine Liste, aus der **genau ein Verb** übergeben wird.
- Annotationen (`PERSON`, `LOC`, `DATE`) mit `find_span()`.
- Ausgabe im gewünschten Python-Listenformat.

👉 Soll ich die Datumsfunktion gleich so umbauen, dass sie „am {tag}. {Monat} {jahr}“ zurückgibt (natürlichere deutsche Form), oder willst du erstmal bei „im {tag} {Monat} {jahr}“ bleiben?



bist du sicher, dass du Funktion richtig geschrieben hast? def
erzeuge_satz_mit_ollama(name: str, ort: str, datum_str: str,
verb_sample: list) -> str:

Ja, gute Nachfrage 🙌 – lass uns das ganz genau anschauen.

Deine Vorgabe für den Prompt war:

```

python
f"Erstelle einen deutschen Satz über die Person '{name}', "
f"die sich in '{ort}' aufhält oder dort etwas tut. "
f"Verwende exakt diesen Namen: {name}, und exakt diesen Ort: {ort}. "
f"Nutze genau eines der folgenden Verben: {', '.join(verb_sample)}. "
f"Der Satz muss das Datum {datum_str} ent- ten, allerdings nicht am Satzanfang – "
f"es soll möglichst in der Satzmitte oder am Ende erscheinen. "

```

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

Gemeinsam nutzen ...

```
f"es soll möglichst in der Satzmitte oder am Ende erscheinen. "
f"Gib ausschließlich den vollständigen Satz aus – keine Erklärungen, Kommentare oder Ei
```

Das habe ich direkt in die Funktion übernommen. Wichtig:

- `verb_sample` ist eine Liste von Strings → darum benutze ich `", ".join(verb_sample)` im Prompt.
- Wenn du nur ein Verb geben willst, sollte `verb_sample` trotzdem eine Liste sein, z. B. `["arbeiten"]`.

Hier nochmal die Funktion sauber isoliert und minimal testbar:

```
python
def erzeuge_satz_mit_ollama(name: str, ort: str, datum_str: str, verb_sample: list) ->
    prompt = (
        f"Erstelle einen deutschen Satz über die Person '{name}', "
        f"die sich in '{ort}' aufhält oder dort etwas tut. "
        f"Verwende exakt diesen Namen: {name}, und exakt diesen Ort: {ort}. "
        f"Nutze genau eines der folgenden Verben: {' '.join(verb_sample)}. "
        f"Der Satz muss das Datum {datum_str} enthalten, allerdings nicht am Satzanfang "
        f"es soll möglichst in der Satzmitte oder am Ende erscheinen. "
        f"Gib ausschließlich den vollständigen Satz aus – keine Erklärungen, Kommentare "
    )

    response = ollama.generate(
        model=MODEL,
        prompt=prompt,
        options={
            "temperature": TEMPERATURE,
            "top_p": 0.9,
            "num_predict": 128,
        }
    )
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...

```
        "temperature": TEMPERATURE,
        "top_p": 0.9,
        "num_predict": 128,
        **({'seed': SEED} if SEED is not None else {}),
    )

text = response["response"].strip()
# Bereinigen
text = text.replace(" ", "").replace(" ", "").replace("'", "").strip()
if "." in text:
    text = text.split(".", 1)[0].strip() + "."
if not text.endswith("."):
    text += "."
return text
```

✓ Damit passt die Funktion zur Signatur: `name`, `ort`, `datum_str` (alles Strings) und `verb_sample` (Liste von Strings).

☞ Wenn du `verb_sample` immer nur ein Verb übergibst, reicht auch eine Liste mit einem Eintrag.

Möchtest du, dass ich die Signatur so ändere, dass `verb_sample` nur ein `str` ist (also `verb: str`) und im Prompt direkt ausgegeben wird? Das wäre noch schlanker, wenn du wirklich nur ein Verb pro Satz willst.

📄 📌 🗑️ 🔍 ↺ ↻

ich bekomme folgende Sätzen, der Code funktioniert nicht richtig: [("Sophie Chriftia Marie Schädelin Schneckenberger verbrachte ihre Tage in Neuwied New Castle, wo sie sich am 5 Oktober 1889

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

Gemeinsam nutzen ...



ich bekomme folgende Sätzen, der Code funktioniert nicht richtig: [

```

("Sophie Chritiane Marie Schädlein Schneckenberger verbrachte
ihre Tage in Neuwied New Castle, wo sie sich am 5 Oktober 1889
jeden Morgen vor dem Aufstehen noch ein paar Träume vom idealen
Leben erlaubte.", [(0, 49, "PERSON"), (74, 92, "LOC")]),
("Am 1.", []),
("Am 18.", []),
("Am 24.", []),
("Am 16.", []),
("Am 22.", []),
("Am 4.", []),
("Johann Ludolph Fabricius Buholo verbringt den 3.", []),
("Rofine Margarethe Mayer Gambold übernachtete im Salem
Centurion am 16 Juni 1859.", [(0, 31, "PERSON"), (48, 63, "LOC")]),
("Levin Theodor Reichel Beyer startete am 29.", [(0, 27, "PERSON")])
]
```

Nachgedacht für 1m 3s

Guter Fund — das Problem kommt vor allem daher, dass wir bisher am **ersten Punkt** abgeschnitten haben. In deutschen Datumsangaben steht nach dem Tag ein Punkt („am 1. Juni ...“), wodurch recht viele Ausgaben zu „Am 1.“ verstümmelt wurden.

Unten ist ein **robuster, kompletter Code**, der:

- `ollama.generate()` verwendet,
- dein Prompt 1:1 nutzt,
- **nicht** mehr am ersten Punkt abschneidet,
- sicherstellt, dass **Datum nicht am Satzanfang** steht (sonst wird nachgebessert),
- die **Spans** für `PERSON`, `LOC` und `DATE` berechnet – inkl. toleranter Erkennung der Datumsvariante

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

Gemeinsam nutzen ...



- sicherstellt, dass **Datum nicht am Satzanfang** steht (sonst wird nachgebessert),
- die **Spans** für `PERSON`, `LOC` und `DATE` berechnet – inkl. toleranter Erkennung der Datumsvariante (`im -- / am --`, mit/ohne Punkt nach dem Tag).

Du kannst ihn so laufen lassen; wenn du das Datum **genau** in der Form `im {tag} {monat} {jahr}` erzwingen willst, stell `REQUIRE_EXACT_DATE = True`.

```

python
# -*- coding: utf-8 -*-
import re
import random
import ollama

MODEL = "llama3"
TEMPERATURE = 0.6
SEED = 42
NUM_PREDICT = 256
MAX_RETRIES = 2 # wie oft wir nachbessern, wenn Bedingungen verletzt sind
REQUIRE_EXACT_DATE = False # auf True setzen, wenn exakt die String-Form verlangt ist

# -----
# Hilfsfunktionen (wie vorgegeben + Ergänzungen)
# -----

def load_list_from_file(filename):
    with open(filename, 'r', encoding='utf-8') as file:
        return [line.strip() for line in file if line.strip()]

def generate_date(raw=False):
    tag = random.choice(datum_list)
    month = random.choice(month_list)
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

[Gemeinsam nutzen](#) ...

```
def generate_date(raw=False):
    tag = random.choice(datum_list)
    monat = random.choice(monat_list)
    jahr = random.choice(jahr_list)
    return (tag, monat, jahr) if raw else f"im {tag} {monat} {jahr}"

def find_span(text, target):
    start = text.find(target)
    return (start, start + len(target)) if start != -1 else None

def normalize_ws(s: str) -> str:
    return re.sub(r"\s+", " ", s).strip()

def starts_with_date(text: str) -> bool:
    """
    Prüft, ob der Satz mit einem Datum wie 'am 1.' oder 'im 1.' beginnt.
    Wir lassen führende Anführungszeichen/Leerzeichen außer Acht.
    """
    t = text.lstrip(' "').lower()
    # am|im + Zahl (mit/ohne Punkt) + Monat
    return bool(re.match(r"^(am|im)\s+\d+\.?\s+\w+", t))

def date_variants(tag: str, monat: str, jahr: str):
    """
    Erzeugt gängige Varianten der gleichen Datumsaussage.
    Beispiele:
        im 5 Oktober 1889
        am 5 Oktober 1889
        im 5. Oktober 1889
        am 5. Oktober 1889
    """
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

[Gemeinsam nutzen](#) ...

```
im 5. Oktober 1889
am 5. Oktober 1889
"""

tag_clean = normalize_ws(tag)
monat_clean = normalize_ws(monat)
jahr_clean = normalize_ws(jahr)

# Tag als Ziffer extrahieren, um Varianten mit/ohne Punkt bauen zu können
m = re.search(r"\d+", tag_clean)
day_digits = m.group(0) if m else tag_clean

# hat der Tag schon einen Punkt?
has_dot = bool(re.search(r"\d+\.", tag_clean))

candidates = set()
for prep in ("im", "am"):
    # ohne Punkt
    candidates.add(f"{prep} {day_digits} {monat_clean} {jahr_clean}")
    # mit Punkt
    candidates.add(f"{prep} {day_digits}. {monat_clean} {jahr_clean}")
# falls im Tag selbst noch weitere Zusätze waren, nimm auch die rohe Variante
candidates.add(f"im {tag_clean} {monat_clean} {jahr_clean}")
candidates.add(f"am {tag_clean} {monat_clean} {jahr_clean}")

# normalisiert zurückgeben (Duplikate entfernt)
return [normalize_ws(c) for c in candidates]

def find_date_span(text: str, tag: str, monat: str, jahr: str, preferred: str = None):
    """
    Sucht tolerant nach der Datumsangabe i-Text.
    - Falls preferred (z. B. 'im 5 Oktober 1889') gegeben ist, versuchen wir zuerst das
    - Ansonsten probieren wir tvnische Varianten (im/am... mit/ohne Punkt).
    """
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

Gemeinsam nutzen ...



Sucht tolerant nach der Datumsangabe im Text.
 - Falls preferred (z. B. 'im 5 Oktober 1889') gegeben ist, versuchen wir zuerst das
 - Ansonsten probieren wir typische Varianten (im/am, mit/ohne Punkt).
 Gibt (matched_text, (start, end)) oder (None, None) zurück.

```

***
text_norm = text # nicht lowercased, da wir exakte Spans brauchen
# bevorzugt zuerst die gewünschte String-Form
variants = []
if preferred:
    variants.append(normalize_ws(preferred))
variants.extend(date_variants(tag, monat, jahr))

seen = set()
for v in variants:
    if v in seen:
        continue
    seen.add(v)
    idx = text_norm.find(v)
    if idx != -1:
        return v, (idx, idx + len(v))
return None, None

```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

Gemeinsam nutzen ...



```

person_list_1 = load_list_from_file('input/person_list.txt')
person_list_2 = load_list_from_file('input/nachname_list.txt') # neu eingegeben
ortsname_list_1 = load_list_from_file('input/ortsname_list.txt')
ortsname_list_2 = load_list_from_file('input/ortsname_SA_E_list.txt') # neu eingegeben
jahr_list = load_list_from_file("input/jahr_list.txt")
monat_list = load_list_from_file("input/monat_list.txt")
datum_list = load_list_from_file("input/datum_list.txt")
verb_list = load_list_from_file("input/verb_list.txt")

# -----
# Satzgenerierung mit ollama.generate()
# -----

def build_prompt(name: str, ort: str, datum_str: str, verb_sample: list) -> str:
    return (
        f"Erstelle einen deutschen Satz über die Person '{name}', "
        f"die sich in '{ort}' aufhält oder dort etwas tut. "
        f"Verwende exakt diesen Namen: {name}, und exakt diesen Ort: {ort}. "
        f"Nutze genau eines der folgenden Verben: {' '.join(verb_sample)}. "
        f"Der Satz muss das Datum {datum_str} enthalten, allerdings nicht am Satzanfang "
        f"es soll möglichst in der Satzmitte oder am Ende erscheinen. "
        f"Gib ausschließlich den vollständigen Satz aus – keine Erklärungen, Kommentare "
    )

def erzeuge_satz_mit_ollama(name: str, ort: str, datum_str: str, tag: str, monat: str,
    """
    Generiert einen Satz, validiert:
    - Name und Ort exakt enthalten
    - Datum enthalten (tolerant am/am + Punktvariante) und nicht am Satzanfang
    - Genau ein Satz (heuristisch)
    Bei Verstoß bis MAX_RETRIES nachbessern.
    """

```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

Gemeinsam nutzen ...



```
def erzeuge_satz_mit_ollama(name: str, ort: str, datum_str: str, t: Kopieren Bearbeiten
    """
    Generiert einen Satz, validiert:
    - Name und Ort exakt enthalten
    - Datum enthalten (tolerant am/am + Punktvariante) und nicht am Satzanfang
    - Genau ein Satz (heuristisch)
    Bei Verstoß bis MAX_RETRIES nachbessern.
    """
    if SEED is not None:
        random.seed(SEED)

    prompt = build_prompt(name, ort, datum_str, verb_sample)

    def call_model(p):
        resp = ollama.generate(
            model=MODEL,
            prompt=p,
            options={
                "temperature": TEMPERATURE,
                "top_p": 0.9,
                "num_predict": NUM_PREDICT,
                **({"seed": SEED} if SEED is not None else {}),
            }
        )
        txt = resp["response"]
        # sanft bereinigen, aber NICHT am ersten Punkt abschneiden!
        txt = normalize_ws(txt.replace(" ", "").replace(" ", "").replace("'", ""))
        # einen Endpunkt sicherstellen
        if not re.search(r"[!?]", txt):
            txt = txt.rstrip(" !?") + "."
        return txt
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

Gemeinsam nutzen ...



```
txt = txt.rstrip(" !?") + "."
return txt

text = call_model(prompt)

# Validierung + ggf. Reprompts
retries = 0
while retries < MAX_RETRIES:
    has_name = (name in text)
    has_loc = (ort in text)
    matched_date, span_date = find_date_span(text, tag, monat, jahr, preferred=datum)
    date_ok = matched_date is not None
    date_not_start = not starts_with_date(text)

    # Heuristik: "genau ein Satz" - wir erlauben Punkte innerhalb von Datums-Tagen
    # Zähle Satz-Endzeichen, aber ignoriere das Muster 'digit.' ohne folgendes Leer
    # Diese Heuristik reicht für unseren Zweck gut aus.
    ends = re.findall(r"[!?]", text)
    too_many_sentences = len(ends) > 1 and bool(re.search(r"[!?]\s+[A-ZÄÖÜ]", text))

    if has_name and has_loc and date_ok and date_not_start and not too_many_sentences:
        break # alles erfüllt

    hints = []
    if not has_name: hints.append("Der Satz enthält den exakten Namen nicht.")
    if not has_loc: hints.append("Der Satz enthält den exakten Ort nicht.")
    if not date_ok: hints.append("Das Datum wurde nicht erkannt.")
    if not date_not_start: hints.append("Das Datum steht am Satzanfang.")
    if too_many_sentences: hints.append("Mehr als ein Satz erkannt.")

    # Nachbesserungs-Prompt
    repair_prompt = build_prompt(name, ort, datum_str, verb_sample) + (
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...



```
# Nachbesserungs-Prompt
repair_prompt = build_prompt(name, ort, datum_str, verb_sample) + (
    "\nAchte unbedingt darauf, NICHT mit dem Datum zu beginnen (nicht mit 'am'
    "und gib genau EINEN vollständigen Satz aus."
)
text = call_model(repair_prompt)
retries += 1

return text

# -----
# Mehrere Sätze generieren + Annotation
# -----

def main(anzahl=10):
    if SEED is not None:
        random.seed(SEED)

    samples = []

    for _ in range(anzahl):
        # Person: Vor- + Nachname
        name = f"{random.choice(person_list_1)} {random.choice(person_list_2)}"
        # Ort: Hauptliste + Zusatz
        ort = f"{random.choice(ortsname_list_1)} {random.choice(ortsname_list_2)}"
        # Datumsteile + bevorzugte String-Form
        tag, monat, jahr = generate_date(raw=True)
        datum_str = f"im {tag} {monat} {jahr}"

        # Verb: gib dem Modell 2-3 Optionen, soll aber nur EINE nutzen
        verb_sample = random.sample(verb_list, k=min(3, len(verb_list)))
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...



```
# Verb: gib dem Modell 2-3 Optionen, soll aber nur EINE nutzen
verb_sample = random.sample(verb_list, k=min(3, len(verb_list)))

satz = erzeuge_satz_mit_ollama(name, ort, datum_str, tag, monat, jahr, verb_sample)

# ---- Spans berechnen ----
spans = []

# PERSON
sp = find_span(satz, name)
if sp: spans.append((sp[0], sp[1], "PERSON"))

# LOC
sp = find_span(satz, ort)
if sp: spans.append((sp[0], sp[1], "LOC"))

# DATE (tolerant)
if REQUIRE_EXACT_DATE:
    sp = find_span(satz, datum_str)
    if sp: spans.append((sp[0], sp[1], "DATE"))
else:
    matched_date, span = find_date_span(satz, tag, monat, jahr, preferred=datum)
    if span: spans.append((span[0], span[1], "DATE"))

samples.append((satz, spans))

# ---- Ausgabe im gewünschten Format ----
print("")
for i, (text, spans) in enumerate(samples):
    # Escape für doppelte Anführungszeichen in der Darstellung
    txt = text.replace("\\", "\\").replace("'", "\'")
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[Gemeinsam nutzen](#) ...


```
# ---- Ausgabe im gewünschten Format ----
print("")
for i, (text, spans) in enumerate(samples):
    # Escape für doppelte Anführungszeichen in der Darstellung
    txt = text.replace("\\", "\\\\").replace("'", "\\'")
    spans_str = ", ".join(f"({s}, {e}, \"{lbl}\")" for (s, e, lbl) in spans)
    comma = ", " if i < len(samples) - 1 else ""
    print(f'  ({txt}, [{spans_str}]){comma}')
print("")

# -----

if __name__ == "__main__":
    main(anzahl=10)
```

Was sich konkret ändert:

- Kein Abschneiden mehr am ersten Punkt → keine „Am 1.“-Fragmente.
- Validierung + Reprompt, wenn das Datum am Satzanfang steht.
- Tolerante Datumserkennung (am/im, mit/ohne Punkt), damit DATE -Spans zuverlässig gefunden werden.
- Längere Vorhersage (num_predict=256) für vollständige Sätze.

Wenn du strikt „im {Tag} {Monat} {Jahr}“ im Text erzwingen willst, setz `REQUIRE_EXACT_DATE = True`.

ich möchte diese Funktionen nicht haben: def normalize_ws(s: str) -> str:

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[Gemeinsam nutzen](#) ...


ich möchte diese Funktionen nicht haben: def normalize_ws(s: str) -> str:

```
return re.sub(r"[s+", " ", s).strip()
```

def starts_with_date(text: str) -> bool:

```
"""
Prüft, ob der Satz mit einem Datum wie 'am 1.' oder 'im 1.' beginnt.
Wir lassen führende Anführungszeichen/Leerzeichen außer Acht.
"""
```

```
t = text.lstrip(' " ').lower()
# am|im + Zahl (mit/ohne Punkt) + Monat
return bool(re.match(r"^(am|im)\s+\d+\.?\s+(w+)", t))
```

def date_variants(tag: str, monat: str, jahr: str):

```
"""
Erzeugt gängige Varianten der gleichen Datumsaussage.
Beispiele:
im 5 Oktober 1889
am 5 Oktober 1889
im 5. Oktober 1889
am 5. Oktober 1889
"""
```

```
tag_clean = normalize_ws(tag)
monat_clean = normalize_ws(monat)
jahr_clean = normalize_ws(jahr)
```

Tag als Ziffer extrahieren, um Varianten mit/ohne Punkt bauen zu können

```
m = re.search(r"^\d+", tag_clean)
day_digits = m.group(0) if m else tag_clean
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

↑ Gemeinsam nutzen ...



```
# Tag als Ziffer extrahieren, um Varianten mit/ohne Punkt bauen zu
können
m = re.search(r"\d+", tag_clean)
day_digits = m.group(0) if m else tag_clean

# hat der Tag schon einen Punkt?
has_dot = bool(re.search(r"\d+\.", tag_clean))

candidates = set()
for prep in ("im", "am"):
    # ohne Punkt
    candidates.add(f"{prep} {day_digits} {monat_clean}
{jahr_clean}")
    # mit Punkt
    candidates.add(f"{prep} {day_digits}. {monat_clean}
{jahr_clean}")
# falls im Tag selbst noch weitere Zusätze waren, nimm auch die
rohe Variante
candidates.add(f"im {tag_clean} {monat_clean} {jahr_clean}")
candidates.add(f"am {tag_clean} {monat_clean} {jahr_clean}")

# normalisiert zurückgeben (Duplikate entfernt)
return [normalize_ws(c) for c in candidates]

def find_date_span(text: str, tag: str, monat: str, jahr: str, preferred:
str = None):
    """
    Sucht tolerant nach der Datumsangabe im Text.
    - Falls preferred (z. B. 'im 5 Oktober 1889') gegeben ist, versuchen
    wir zuerst das.
    - Ansonsten probieren wir typische Varianten (im/am, mit/ohne
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

↑ Gemeinsam nutzen ...



```
def find_date_span(text: str, tag: str, monat: str, jahr: str, preferred:
str = None):
    """
    Sucht tolerant nach der Datumsangabe im Text.
    - Falls preferred (z. B. 'im 5 Oktober 1889') gegeben ist, versuchen
    wir zuerst das.
    - Ansonsten probieren wir typische Varianten (im/am, mit/ohne
    Punkt).
    Gibt (matched_text, (start, end)) oder (None, None) zurück.
    """
    text_norm = text # nicht lowercased, da wir exakte Spans
    brauchen
    # bevorzugt zuerst die gewünschte String-Form
    variants = []
    if preferred:
        variants.append(normalize_ws(preferred))
        variants.extend(date_variants(tag, monat, jahr))

    seen = set()
    for v in variants:
        if v in seen:
            continue
        seen.add(v)
        idx = text_norm.find(v)
        if idx != -1:
            return v, (idx, idx + len(v))
    return None, None
```

Verstanden 🙌 — du willst also die Zusatzfunktion `normalize_ws`, `starts_with_date`,
`date_variants` und `find_date_span` nicht haben.

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

Gemeinsam nutzen ...



Verstanden 🙌 — du willst also die Zusatzfunktionen `normalize_ws`, `starts_with_date`, `date_variants` und `find_date_span` nicht haben.
Dann halte ich den Code schlanker: wir arbeiten nur mit den Basisfunktionen, die du schon angegeben hast (`load_list_from_file`, `generate_date`, `find_span`) und suchen das Datum exakt so, wie wir es im Prompt übergeben.

Hier der aufgeräumte, vollständige Code ohne die von dir unerwünschten Funktionen:

```
python
# -*- coding: utf-8 -*-
import random
import ollama

MODEL = "llama3"
TEMPERATURE = 0.6
SEED = 42
NUM_PREDICT = 256

# -----
# Hilfsfunktionen
# -----

def load_list_from_file(filename):
    with open(filename, 'r', encoding='utf-8') as file:
        return [line.strip() for line in file if line.strip()]

def generate_date(raw=False):
    tag = random.choice(datum_list)
    monat = random.choice(monat_list)
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

Gemeinsam nutzen ...



```
def generate_date(raw=False):
    tag = random.choice(datum_list)
    monat = random.choice(monat_list)
    jahr = random.choice(jahr_list)
    return (tag, monat, jahr) if raw else f"im {tag} {monat} {jahr}"

def find_span(text, target):
    start = text.find(target)
    return (start, start + len(target)) if start != -1 else None

# -----
# Daten laden
# -----

person_list_1 = load_list_from_file('input/person_list.txt')
person_list_2 = load_list_from_file('input/nachname_list.txt')
ortsname_list_1 = load_list_from_file('input/ortsname_list.txt')
ortsname_list_2 = load_list_from_file('input/ortsname_SA_E_list.txt')
jahr_list = load_list_from_file("input/jahr_list.txt")
monat_list = load_list_from_file("input/monat_list.txt")
datum_list = load_list_from_file("input/datum_list.txt")
verb_list = load_list_from_file("input/verb_list.txt")

# -----
# Satzgenerierung
# -----

def erzeuge_satz_mit_ollama(name: str, ort: str, datum_str: str, verb_sample: list) ->
    prompt = (
        f"Erstelle einen deutschen Satz ü die Person '{name}', "
        f"die sich in '{ort}' aufhält oder dort etwas tut. "
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

[Gemeinsam nutzen](#) ...


```
def erzeuge_satz_mit_ollama(name: str, ort: str, datum_str: str, v: bool, k: bool, b: bool):
    prompt = (
        f"Erstelle einen deutschen Satz über die Person '{name}', "
        f"die sich in '{ort}' aufhält oder dort etwas tut. "
        f"Verwende exakt diesen Namen: {name}, und exakt diesen Ort: {ort}. "
        f"Nutze genau eines der folgenden Verben: {', '.join(verb_sample)}. "
        f"Der Satz muss das Datum {datum_str} enthalten, allerdings nicht am Satzanfang "
        f"es soll möglichst in der Satzmitte oder am Ende erscheinen. "
        f"Gib ausschließlich den vollständigen Satz aus – keine Erklärungen, Kommentare "
    )

    response = ollama.generate(
        model=MODEL,
        prompt=prompt,
        options={
            "temperature": TEMPERATURE,
            "top_p": 0.9,
            "num_predict": NUM_PREDICT,
            **({"seed": SEED} if SEED is not None else {}),
        }
    )

    text = response["response"].strip()
    text = text.replace(" ", "").replace("(", "").replace(")", "").strip()
    # Einen Punkt sicherstellen
    if not text.endswith("."):
        text += "."
    return text

# -----
# Mehrere Sätze generieren + Annotation
# -----
```

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

[Gemeinsam nutzen](#) ...


```
# -----
# Mehrere Sätze generieren + Annotation
# -----

def main(anzahl=10):
    if SEED is not None:
        random.seed(SEED)

    samples = []

    for _ in range(anzahl):
        # Person: Vor- + Nachname
        name = f"{random.choice(person_list_1)} {random.choice(person_list_2)}"
        # Ort: Hauptliste + Zusatz
        ort = f"{random.choice(ortsname_list_1)} {random.choice(ortsname_list_2)}"
        # Datum
        tag, monat, jahr = generate_date(raw=True)
        datum_str = f"im {tag} {monat} {jahr}"
        # Verb (nur eins)
        verb_sample = [random.choice(verb_list)]

        satz = erzeuge_satz_mit_ollama(name, ort, datum_str, verb_sample)

        # Spans berechnen
        spans = []
        for surface, label in [(name, "PERSON"), (ort, "LOC"), (datum_str, "DATE")]:
            sp = find_span(satz, surface)
            if sp:
                spans.append((sp[0], sp[1], label))

        samples.append((satz, spans))

# -----
```

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...



```
# Ausgabe im gewünschten Format
print("[")
for i, (text, spans) in enumerate(samples):
    spans_str = ", ".join(f"({s}, {e}, \"{lbl}\")" for (s, e, lbl) in spans)
    comma = ", " if i < len(samples)-1 else ""
    print(f'  ({text}, [{spans_str}]){comma}')
print("]")

# -----

if __name__ == "__main__":
    main(anzahl=10)
```

👉 In dieser Variante:

- Das Datum wird **exakt** in der Form "im {tag} {monat} {jahr}" erzeugt und gesucht.
- Keine Toleranzen (am, mit/ohne Punkt) mehr.
- Damit muss das Modell auch wirklich diese Formulierung übernehmen, sonst gibt es kein DATE-Span.

Möchtest du, dass ich das Modell stärker zwingen, **immer exakt** im {tag} {monat} {jahr} zu verwenden (z. B. mit einem zusätzlichen Satz im Prompt „Verwende das Datum exakt so“)?



leider gibt es kein Datum in der Sätzen

Nachgedacht für 6s

Guter Hinweis – dann erzwingen wir das Datum **wortwörtlich** im Satz und reprompten, falls es fehlt (oder am Satzanfang steht).

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#)

ChatGPT 5

Gemeinsam nutzen ...



Guter Hinweis – dann erzwingen wir das Datum **wortwörtlich** im Satz und reprompten, falls es fehlt (oder am Satzanfang steht).

Unten ist der **vollständige Code** mit `ollama.generate()` und einer kleinen Validierungsschleife (ohne die von dir abgelehnten Zusatzfunktionen).

```
python

# -*- coding: utf-8 -*-
import random
import ollama

MODEL = "llama3"
TEMPERATURE = 0.6
SEED = 42
NUM_PREDICT = 256
MAX_RETRIES = 2 # wie oft wir nachbessern, wenn das Datum fehlt oder am Satzanfang steht

# -----
# Hilfsfunktionen (von dir vorgegeben)
# -----

def load_list_from_file(filename):
    with open(filename, 'r', encoding='utf-8') as file:
        return [line.strip() for line in file if line.strip()]

def generate_date(raw=False):
    tag = random.choice(datum_list)
    monat = random.choice(monat_list)
    jahr = random.choice(jahr_list)
    return (tag, monat, jahr) if raw else "im {tag} {monat} {jahr}"
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#)

ChatGPT 5 ▾

↑ Gemeinsam nutzen ...

```
def find_span(text, target):
    start = text.find(target)
    return (start, start + len(target)) if start != -1 else None

# -----
# Daten laden
# -----

person_list_1 = load_list_from_file('input/person_list.txt')
person_list_2 = load_list_from_file('input/nachname_list.txt') # neu eingebe
ortsname_list_1 = load_list_from_file('input/ortsname_list.txt')
ortsname_list_2 = load_list_from_file('input/ortsname_SA_E_list.txt') # neu eingebe
jahr_list = load_list_from_file("input/jahr_list.txt")
monat_list = load_list_from_file("input/monat_list.txt")
datum_list = load_list_from_file("input/datum_list.txt")
verb_list = load_list_from_file("input/verb_list.txt")

# -----
# Prompt & Generierung mit ollama.generate()
# -----

def build_prompt(name: str, ort: str, datum_str: str, verb_sample: list) -> str:
    # Dein gewünschter Prompt 1:1
    return (
        f"Erstelle einen deutschen Satz über die Person '{name}', "
        f"die sich in '{ort}' aufhält oder dort etwas tut. "
        f"Verwende exakt diesen Namen: {name}, und exakt diesen Ort: {ort}. "
        f"Nutze genau eines der folgenden Verben: {', '.join(verb_sample)}. "
        f"Der Satz muss das Datum {datum_str} enthalten, allerdings nicht am Satzanfang "
        f"es soll möglichst in der Satzmitte oder am Ende erscheinen. "
        f"Gib ausschließlich den vollständigen Satz aus – keine Erklärungen, Kommentare"
    )
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

↑ Gemeinsam nutzen ...

```
def generate_sentence(name: str, ort: str, datum_str: str, verb_sa: list) -> str:
    """Generiert einen Satz; wenn das Datum fehlt oder am Satzanfang steht, wird streng
    if SEED is not None:
        random.seed(SEED)

    def call_model(prompt: str) -> str:
        resp = ollama.generate(
            model=MODEL,
            prompt=prompt,
            options={
                "temperature": TEMPERATURE,
                "top_p": 0.9,
                "num_predict": NUM_PREDICT,
                **({"seed": SEED} if SEED is not None else {}),
            }
        )
        text = resp["response"].replace("\n", "").replace(" ", "").replace("'", "").strip()
        if not text.endswith("."):
            text += "."
        return text

    prompt = build_prompt(name, ort, datum_str, verb_sample)
    text = call_model(prompt)

    retries = 0
    while retries < MAX_RETRIES:
        # Muss das exakte Datum enthalten
        has_date = (datum_str in text)
        # Datum darf nicht am Satzanfang stehen (einfacher Check, ohne Hilfsfunktionen)
        starts_with_date = text.lstrip(' ').startswith(datum_str)

        if has_date and not starts_with_date and (retries < MAX_RETRIES):
            retries += 1
            continue
        return text
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...



```

while retries < MAX_RETRIES:
    # Muss das exakte Datum enthalten
    has_date = (datum_str in text)
    # Datum darf nicht am Satzanfang stehen (einfacher Check, ohne Hilfsfunktionen)
    starts_with_date = text.lstrip(' ').startswith(datum_str)

    if has_date and not starts_with_date and (name in text) and (ort in text):
        break

    # Strengerer Reprompt
    strict_prompt = (
        build_prompt(name, ort, datum_str, verb_sample)
        + "\nWICHTIG: Füge den Datumsstring exakt wortwörtlich ein: "
        + f'"{datum_str}". Beginne NICHT mit dem Datum. Gib genau einen vollständig
    )
    text = call_model(strict_prompt)
    retries += 1

return text

# -----
# Mehrere Sätze generieren + Annotation
# -----

def main(anzahl=10):
    if SEED is not None:
        random.seed(SEED)

    samples = []

    for _ in range(anzahl):
        # Person: Vor- + Nachname

```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...



```

for _ in range(anzahl):
    # Person: Vor- + Nachname
    name = f'{random.choice(person_list_1)} {random.choice(person_list_2)}'
    # Ort: Hauptliste + Zusatz
    ort = f'{random.choice(ortsname_list_1)} {random.choice(ortsname_list_2)}'
    # Datum (exakt so, wie wir es verlangen und später als Span suchen)
    tag, monat, jahr = generate_date(raw=True)
    datum_str = f'{tag} {monat} {jahr}'
    # Verboptionen (Modell darf genau eines nutzen)
    verb_sample = [random.choice(verb_list)]

    satz = generate_sentence(name, ort, datum_str, verb_sample)

    # Spans berechnen (exakte Stringsuche)
    spans = []
    for surface, label in [(name, "PERSON"), (ort, "LOC"), (datum_str, "DATE")]:
        sp = find_span(satz, surface)
        if sp:
            spans.append((sp[0], sp[1], label))

    samples.append((satz, spans))

# Ausgabe im gewünschten Format
print("[")
for i, (text, spans) in enumerate(samples):
    # einfache Escape-Handhabung
    safe_text = text.replace("\\", "\\\").replace("'", "\\'")
    spans_str = ", ".join(f'({s}, {e}, \"{lbl}\")' for (s, e, lbl) in spans)
    comma = "," if i < len(samples) - 1 else ""
    print(f'  ({safe_text}, [{spans_str}]){comma}')
print("]")

```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

Seitenleiste öffnen

Gemeinsam nutzen ...



```
if __name__ == "__main__":
    main(anzahl=10)
```

Kopieren Bearbeiten

Was geändert wurde:

- Strenger Prompt + **Reprompt-Schleife** (max. 2 Versuche), bis das exakte `datum_str` im Satz vorkommt und **nicht** am Satzanfang steht.
- Datumsuche bleibt einfach über `find_span(satz, datum_str)` (exakte Übereinstimmung).
- Keine der zuvor unerwünschten Zusatzfunktionen ist enthalten.



Kannst du bitte die Funktion überprüfen:

```
def generate_examples_type1(model_name="llama3", max_retries=6,
    verbs_per_prompt=10):
    random.shuffle(person_list_1)
    random.shuffle(ortsname_list_1)
    ort_cycle = (ortsname_list_1 * ((len(person_list_1) //
    len(ortsname_list_1)) + 1))[:len(person_list_1)]

    count_ok = 0
    for name, ort in zip(person_list_1, ort_cycle):
        attempts = 0
        while attempts < max_retries:
            tag, monat, jahr = generate_date(raw=True)
            datum_str = f"im {tag} {monat} {jahr}"
            verb_sample = random.sample(verb_list,
            min(verbs_per_p
            pt, len(verb_list)))
            prompt = build_prompt_type1(name, ort, verb_sample,
            datum_str)
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

Gemeinsam nutzen ...



```
count_ok = 0
for name, ort in zip(person_list_1, ort_cycle):
    attempts = 0
    while attempts < max_retries:
        tag, monat, jahr = generate_date(raw=True)
        datum_str = f"im {tag} {monat} {jahr}"
        verb_sample = random.sample(verb_list,
        min(verbs_per_prompt, len(verb_list)))
        prompt = build_prompt_type1(name, ort, verb_sample,
        datum_str)

    try:
        result = ollama.generate(model=model_name,
        prompt=prompt)
        satz = result['response'].strip()

        if name in satz and ort in satz and jahr in satz:

            span = find_span(satz, name)
            if span: entities.append([span[0], span[1], "PERSON"])

            span = find_span(satz, ort)
            if span: entities.append([span[0], span[1], "LOC"])

            full = f"{tag} {monat} {jahr}"
            span = find_span(satz, full) or find_span(satz, jahr)
            if span: entities.append([span[0], span[1], "DATE"])

        if entities:
            yie, itz, entities
            print(f"[Typ1] {satz}")
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen

```

print([typ1[satz]
break

except Exception as e:
    print(f"Fehler: {e}")
    time.sleep(1)
    attempts += 1
print(f"Typ1: {count_ok} Beispiele erfolgreich erzeugt.")
print(f"Typ1: {count_ok} Beispiele erfolgreich erzeugt.")

```

Nachgedacht für 9s

Du hast da ein paar fiese, aber typische Bugs drin. Hier ist eine **korrigierte, robuste** Version von `generate_examples_type1(...)`, die zu deinem bisherigen Code passt (keine Zusatzfunktionen wie `normalize_ws` etc.). Ich liste erst die Fixes und gebe dann die **drop-in**-Funktion.

Was schief lief (kurz)

- `entities` wurde nie initialisiert → sammelt nichts / `NameError`.
- `break` ist falsch eingerückt → bricht nicht die richtige Schleife ab.
- `attempts` wurde bei "inhaltlich fehlgeschlagenem" Versuch nicht erhöht → potenziell Endlosschleife.
- Datum nur über `jahr` in `satz` geprüft → viel zu locker; wir erzwingen jetzt **exakt** `datum_str`.
- Datum evtl. am Satzbeginn → einfache Prüfung eingebaut (ohne Extra-Helfer).
- Kein Punkt ans Satzende erzwingen → fragile Span-Offsets bei Nachverarbeitung.
- Doppelte Abschluss-Prints → bereinigt.

Drop-in: korrigierte Funktion

Voraussetzungen: `build_prompt_type1(name, ort, verb_sample, datum_str)`, `generate_date`, `find_span`, sowie die Listen `person_list_1`, `sname_list_1`, `verb_list`, `datum_list`, `monat_list`, `jahr_list` existieren (wie in deinem Projekt). Außerdem: `import random, time`.

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen

Drop-in: korrigierte Funktion

Voraussetzungen: `build_prompt_type1(name, ort, verb_sample, datum_str)`, `generate_date`, `find_span`, sowie die Listen `person_list_1`, `ortsname_list_1`, `verb_list`, `datum_list`, `monat_list`, `jahr_list` existieren (wie in deinem Projekt). Außerdem: `import random, time`, `ollama`.

```

python
import time
import random
import ollama

def generate_examples_type1(model_name="llama3", max_retries=6, verbs_per_prompt=10):
    """
    Erzeugt (satz, entities)-Paare als Generator.
    Bedingungen pro Satz:
    - exakter Personennamen und exakter Ort enthalten
    - exakter Datumsstring (z. B. 'im 12 Mai 1998') enthalten
    - Datum NICHT am Satzbeginn
    """
    # etwas Durchmischen für Varianz
    random.shuffle(person_list_1)
    random.shuffle(ortsname_list_1)

    if not person_list_1 or not ortsname_list_1:
        print("[Typ1] Leere Personen- oder Ortsliste.")
        return

    # Orte auf Personenlänge "auffüllen"
    ort_cycle = (ortsname_list_1 * ((len(person_list_1) // len(ortsname_list_1) + 1)))

```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

[Gemeinsam nutzen](#) ...


```
# Orte auf Personenlänge "auffüllen"
ort_cycle = (ortsname_list_1 * ((len(person_list_1) // len(ortsname_list_1) + 1)))

count_ok = 0

for name, ort in zip(person_list_1, ort_cycle):
    attempts = 0

    while attempts < max_retries:
        # Datum exakt so erzeugen, wie es auch gefordert und später gesucht wird
        tag, monat, jahr = generate_date(raw=True)
        datum_str = f"im {tag} {monat} {jahr}"

        # Verboptionen (Modell darf genau eins nutzen, Prompt listet mehrere)
        if verbs_per_prompt <= 0:
            verb_sample = [random.choice(verb_list)]
        else:
            verb_sample = random.sample(verb_list, min(verbs_per_prompt, len(verb_list)))

        prompt = build_prompt_type1(name, ort, verb_sample, datum_str)

        try:
            result = ollama.generate(
                model=model_name,
                prompt=prompt,
                options={
                    "temperature": 0.6,
                    "top_p": 0.9,
                    "num_predict": 256,
                }
            )
```

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

[Gemeinsam nutzen](#) ...


```
try:
    result = ollama.generate(
        model=model_name,
        prompt=prompt,
        options={
            "temperature": 0.6,
            "top_p": 0.9,
            "num_predict": 256,
        }
    )

    satz = (result.get("response") or "").replace("\n", "").replace(" ", "")
    if not satz.endswith("."):
        satz += "."

    # --- Validierung ---
    has_name = (name in satz)
    has_loc = (ort in satz)
    # Exakt den Datumsstring fordern (nicht nur das Jahr)
    has_date = (datum_str in satz)
    # Datum darf nicht am Satzanfang stehen (einfach, ohne Zusatzfunktionen)
    starts_with_date = satz.lstrip(' ').startswith(datum_str)

    if has_name and has_loc and has_date and not starts_with_date:
        # Entities pro erfolgreichem Satz neu initialisieren
        entities = []

        sp = find_span(satz, name)
        if sp: entities.append([sp[0], sp[1], "PERSON"])

        sp = find_span(satz, datum_str)
        if sp: entities.append([sp[0], sp[1], "LOC"])
```

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[Gemeinsam nutzen](#) ...


```

if entities:
    yield satz, entities
    print(f"[Typ1] {satz}")
    count_ok += 1
    break # raus aus der Retry-Schleife für diese Person/Ort
else:
    # inhaltlich fehlgeschlagen -> nächster Versuch
    attempts += 1

except Exception as e:
    print(f"Fehler: {e}")
    time.sleep(1)
    attempts += 1

print(f"Typ1: {count_ok} Beispiele erfolgreich erzeugt.")

```

Warum diese Checks?

- **Exaktes Datum:** Wir suchen `datum_str` (z. B. `im 5 Oktober 1889`) 1:1 im Text → vermeidet falsche Positivtreffer wie „Am 5.“ oder nur „1889“.
- **Nicht am Satzanfang:** Simple `lstrip(...).startswith(datum_str)` genügt – du wolltest keine Zusatzhelfer.
- **Retries:** Erhöhen wir sowohl bei Fehlern als auch bei inhaltlich ungenügenden Antworten.

Wenn du willst, kann ich dir auch eine kleine `build_prompt_type1(...)`-Funktion mitschicken, die exakt deinen zuletzt gewünschten Prompttext nutzt.

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

5.4.2 Relationsextraktion

ChatGPT 4o

Gemeinsam nutzen

...

ich arbeite mit ollama Model llama3 und LangChain. Ich möchte aus den Sätzen bestimmte Information extrahieren. Dafür habe ich zwei prompts erstellt (system, human). Wie kann ich Beispiele definieren

Super! Du arbeitest mit LangChain, Ollama (LLaMA 3) und willst mithilfe von Prompts und Beispielen (few-shot learning) Informationen aus Texten extrahieren.

Das geht in LangChain sehr gut mit einem FewShotPromptTemplate, kombiniert mit deinen eigenen System- und Human-Prompts.

Ziel: Few-shot Prompt mit Beispielen + LLaMA 3 (Ollama)

Schritt-für-Schritt-Anleitung

1. Importiere die nötigen Bausteine

python

Kopieren

Bearbeiten

```
from langchain.prompts import PromptTemplate, FewShotPromptTemplate
from langchain_core.prompts import ChatPromptTemplate, SystemMessagePromptTemplate, Hum
```

2. Beispielstruktur definieren

Du brauchst ein paar Beispiel-Sätze + erwartete Extraktion. Z.B.:

python

Kopieren

Bearbeiten

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#)

2. Beispielstruktur definieren

Du brauchst ein paar Beispiel-Sätze + erwartete Extraktion. Z.B.:

python

Kopieren

Bearbeiten

```
examples = [
    {
        "input": "Daniel Wilhelm Suhl ging am 30. April 1858 nach Gnadenthal.",
        "output": "Subjekt: Daniel Wilhelm Suhl | Prädikat: ging nach | Objekt: Gnadent
```

3. Erstelle den Beispiel-Prompt

python

Kopieren

Bearbeiten

```
example_prompt = PromptTemplate(
    input_variables=["input", "output"],
    template="Text: {input}\nExtraktion: {output}\n"
```

4. FewShotPromptTemplate erstellen

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#)



ChatGPT 4o

Gemeinsam nutzen ...

4. 📌 FewShotPromptTemplate erstellen

```
python
few_shot_prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=example_prompt,
    prefix="Extrahiere Subjekt, Prädikat, Objekt und Zeitangabe aus folgendem Text:",
    suffix="Text: {text}\nExtraktion:",
    input_variables=["text"]
)
```

5. 🗨️ Prompt in LangChain mit Chat-Modell (z.B. Ollama / LLaMA3) verwenden

```
python
from langchain_ollama import ChatOllama
from langchain.chains import LLMChain

llm = ChatOllama(model="llama3")

chain = LLMChain(llm=llm, prompt=few_shot_prompt)
```

6. 🚀 Ausführen

```
python
# Stelle irgendeine Frage
```

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 4o

Gemeinsam nutzen ...

6. 🚀 Ausführen

```
python
text = "Er versprach am 4. Mai 1857 in Herrnhut zu bleiben."
result = chain.run(text=text)
print(result)
```

🔧 Optional: System- und Human-Prompt getrennt verwenden

Wenn du lieber mit **System-Message + Human-Message** arbeiten willst, kannst du so vorgehen:

```
python
from langchain_core.prompts import ChatPromptTemplate, SystemMessagePromptTemplate, HumanMessagePromptTemplate

system_template = "Du bist ein Extraktionsmodell für historische Texte. Extrahiere Subj"
system_message_prompt = SystemMessagePromptTemplate.from_template(system_template)

human_template = "Text: {text}"
human_message_prompt = HumanMessagePromptTemplate.from_template(human_template)

chat_prompt = ChatPromptTemplate.from_messages([system_message_prompt, human_message_prompt])

chain = LLMChain(llm=llm, prompt=chat_prompt)
```

Aber: Für **Few-Shot Learning** ist die `FewShotPromptTemplate` einfacher und effektiver.

```
python
# Stelle irgendeine Frage
```

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 4o

Gemeinsam nutzen

Ergebnis

Damit kannst du dem LLaMA3-Modell Beispiele geben, wie es Informationen extrahieren soll, und es folgt deinem Muster deutlich besser.

Wenn du willst, kann ich dir helfen, dein Prompt-Design noch weiter zu verfeinern (z. B. mit `pydantic` - Validierung oder JSON-Ausgaben für strukturierte Verarbeitung). Sag einfach Bescheid.

ja bitte

Perfekt – dann zeige ich dir jetzt, wie du:

Informationen strukturiert als JSON aus Texten extrahierst

– mit LangChain, Ollama (LLaMA3), und Pydantic zur Validierung der LLM-Ausgabe.

Ziel:

Du gibst z. B. den Satz:

„Daniel Wilhelm Suhl ging am 30. April 1858 nach Gnadenthal.“

Und bekommst automatisch ein valide strukturiertes Python-Objekt (oder JSON), z. B.:

json

Kopieren Bearbeiten

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

Ziel:

Du gibst z. B. den Satz:

„Daniel Wilhelm Suhl ging am 30. April 1858 nach Gnadenthal.“

Und bekommst automatisch ein valide strukturiertes Python-Objekt (oder JSON), z. B.:

json

Kopieren Bearbeiten

```
{
  "subjekt": "Daniel Wilhelm Suhl",
  "subjekt_type": "person",
  "prädikat": "ging nach",
  "objekt": "Gnadenthal",
  "objekt_type": "location",
  "zeit": "30. April 1858"
}
```

Schritt-für-Schritt

1. Installiere die nötigen Pakete (falls noch nicht geschehen)

bash

Kopieren Bearbeiten

```
pip install langchain langchain-ollama pydantic
```

↓

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 4o

Gemeinsam nutzen ...

2. 🐍 Definiere dein Datenmodell mit pydantic

```
python
from pydantic import BaseModel, Field
from typing import Optional

class ExtractedInfo(BaseModel):
    subjekt: str = Field(description="Subjekt-Entität, z. B. Daniel Wilhelm Suhl, ich,")
    subjekt_type: str = Field(description="Typ des Subjekts, z. B. person, pronoun")
    prädikat: str = Field(description="Beziehung zwischen Subjekt und Objekt")
    objekt: str = Field(description="Objekt-Entität, z. B. Gnadenthal, Aeltesten-Konfer")
    objekt_type: str = Field(description="Typ des Objekts, z. B. location, organisation")
    zeit: Optional[str] = Field(default=None, description="Datum oder Zeitangabe, falls")
```

3. 🗨️ Baue eine prompt -Struktur mit Anweisung + Beispiel

```
python
from langchain.prompts import PromptTemplate
from langchain.output_parsers import PydanticOutputParser

parser = PydanticOutputParser(pydantic_object=ExtractedInfo)

prompt = PromptTemplate(
    template="""
    Extrahiere folgende Informationen aus dem gegebenen Text:
    - Subjekt
    - Typ des Subjekts
    ↓
    + Stelle irgendeine Frage
```

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 4o

Gemeinsam nutzen ...

```
prompt = PromptTemplate(
    template="""
    Extrahiere folgende Informationen aus dem gegebenen Text:
    - Subjekt
    - Typ des Subjekts
    - Prädikat
    - Objekt
    - Typ des Objekts
    - Zeitangabe (falls vorhanden)

    {format_instructions}

    Text: {text}
    """,
    input_variables=["text"],
    partial_variables={"format_instructions": parser.get_format_instructions()},
)
```

4. 🦙 LLaMA 3 (Ollama) einbinden

```
python
from langchain_ollama import ChatOllama
from langchain.chains import LLMChain

llm = ChatOllama(model="llama3")

chain = LLMChain(llm=llm, prompt=prompt)
↓
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 4o ▾

[↑ Gemeinsam nutzen](#) ...

5. 🚀 Aufruf & strukturierte Ausgabe erzeugen

```
python 📄 Kopieren 🛠️ Bearbeiten  
  
text = "Daniel Wilhelm Suhl ging am 30. April 1858 nach Gnadenthal."  
  
# LLM generiert den Roh-Output als Text  
raw_output = chain.run(text=text)  
  
# Mit pydantic-Parser in ein Python-Objekt umwandeln  
structured_result = parser.parse(raw_output)  
  
print(structured_result)  
print(structured_result.json(indent=2))
```

✅ Ausgabe

```
json 📄 Kopieren 🛠️ Bearbeiten  
  
{  
  "subjekt": "Daniel Wilhelm Suhl",  
  "subjekt_type": "person",  
  "prädikat": "ging nach",  
  "objekt": "Gnadenthal",  
  "objekt_type": "location",  
  "zeit": "30. April 1858"  
}
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#)



ChatGPT 4o ▾

[↑ Gemeinsam nutzen](#) ...

```
"objekt": "Gnadenthal",  
"objekt_type": "location",  
"zeit": "30. April 1858"  
}
```

✅ Vorteile dieses Ansatzes:

- Du bekommst **strukturierten Output** direkt vom LLM – kein Regex, kein Parsen nötig.
- **Validierung durch Pydantic**: Wenn das Modell falsches Format liefert, wird ein Fehler geworfen.
- Du kannst die Daten **direkt weiterverarbeiten** (z. B. speichern, analysieren, in TEI/XML integrieren).

📌 Bonus: Beispiel für invalides Ergebnis erkennen

Wenn das Modell z. B. "subjekt_type" vergisst, wirft `pydantic` einen `ValidationError` – das ist extrem hilfreich zur Fehlerkontrolle.

👉 Nächster Schritt?

Wenn du willst, kann ich dir zeigen, wie du:

- Mehrere Sätze gleichzeitig extrahierst
- Den Output automatisch als `.json` speicherst
- Oder das Ganze mit einer GUI/Streamlit verbindest

Sag einfach, was du brauchst.



+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#)





soll ich auch Klassen definieren? Wie hier: `class User(BaseModel):`

```

id: int
id is of type int; the annotation-only declaration tells Pydantic that
this field is required. Strings, bytes, or floats will be coerced to
integers if possible; otherwise an exception will be raised.

name: str = 'John Doe'
signup_ts: datetime | None
tastes: dict[str, PositiveInt]
```

Ja, solche Klassen solltest du definieren, wenn du strukturierte Daten modellieren, validieren oder ausgeben möchtest – besonders bei deinem Fall (Informationsextraktion aus Texten mit LLMs).

Warum Klassen wie `User` mit `pydantic.BaseModel` nützlich sind:

✅ 1. Strukturierte Daten definieren

Du sagst Python (und LLMs!), wie deine Daten aussehen sollen. Beispiel:

```
python
class User(BaseModel):
    id: int
    name: str = 'John Doe'
    signup_ts: datetime | None
    tastes: dict[str, PositiveInt]
```

Das hilft dir und dem Modell zu wissen:

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#)



Das hilft dir und dem Modell zu wissen:

- Was gehört in die Daten?
- Welche Typen sind erlaubt?
- Welche Felder sind optional oder haben Defaults?

✅ 2. Eingaben validieren

Pydantic prüft automatisch:

- Ist `id` wirklich eine ganze Zahl?
- Ist `signup_ts` ein `datetime`-Objekt (oder `None`)?
- Ist `tastes` ein Dictionary mit positiven Ganzzahlen?

✅ 3. Automatisch JSON erzeugen oder auslesen

Du kannst:

```
python
u = User(id="1", tastes={"sweet": 3})
print(u.json(indent=2))
```

Ergebnis:

```
json
{
  "id": 1,
  "name": "John Doe",
  "signup_ts": null,
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#)

ChatGPT 4o

Gemeinsam nutzen ...

```
json
{
  "id": 1,
  "name": "John Doe",
  "signup_ts": null,
  "tastes": {
    "sweet": 3
  }
}
```

Und in deinem Fall (LLM + Extraktion):

Statt `User`, brauchst du z. B.:

```
python
class ExtractedInfo(BaseModel):
    subjekt: str
    subjekt_type: str
    prädikat: str
    objekt: str
    objekt_type: str
    zeit: Optional[str] = None
```

Das hat dieselben Vorteile:

- **Struktur:** Du weißt, welche Felder extrahiert werden sollen.
- **Validierung:** Fehlerhafte oder fehlende Felder werden erkannt.
- **LLM-Integration:** Du kannst `pydantic`-Modelle direkt in LangChain verwenden (z. B. mit `PydanticOutputParser`).

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 4o

Gemeinsam nutzen ...

```
python
class ExtractedInfo(BaseModel):
    subjekt: str
    subjekt_type: str
    prädikat: str
    objekt: str
    objekt_type: str
    zeit: Optional[str] = None
```

Das hat dieselben Vorteile:

- **Struktur:** Du weißt, welche Felder extrahiert werden sollen.
- **Validierung:** Fehlerhafte oder fehlende Felder werden erkannt.
- **LLM-Integration:** Du kannst `pydantic`-Modelle direkt in LangChain verwenden (z. B. mit `PydanticOutputParser`).

TL;DR

Ja, du solltest solche Klassen definieren – das ist **sauberer, sicherer und LLM-freundlich**. Du gewinnst:

- "Klarheit"
- "automatische Validierung"
- "einfachere Weiterverarbeitung (z. B. als JSON, XML etc.)"

Wenn du möchtest, kann ich dir ein **komplettes Template mit mehreren Modellen** (z. B. `ExtractedInfo`, `Person`, `Ort`, ...) aufbauen – je nachdem, wie komplex deine Daten sind.

📄 📌 🔍 🗑️ ↺ ↻

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

Gemeinsam nutzen ...



kannst du bitte überprüfen, ob alle variable benutzt werden, aber
ändere bitte mein Code nicht: import json
from pydantic import BaseModel, Field
from langchain.chat_models import ChatOllama
from langchain.prompts import PromptTemplate,
SystemMessagePromptTemplate, HumanMessagePromptTemplate,
ChatPromptTemplate
from langchain.chains import LLMChain
from langchain.output_parsers import PydanticOutputParser # Lade
Eingabedaten
with open("data/5.4.2_RE/sätze.json", "r", encoding="utf-8") as f:
input_data = json.load(f) # Neue Entity- und Relationstypen
entity_types = ['person', 'location', 'organization', 'date']

relation_types = [
Biografische Basisdaten
"geboren_in", "geboren_am",
"gestorben_in", "gestorben_am",
"begraben_in",

Wohn- und Wirkorte
"wohnhaft_in", "lebte_in", "wirkte_in", "aufenthalt_in",

Bildung und Beruf
"ausgebildet_in", "studierte_an",
"lehre_als", "lehre_bei",
"tätig_als", "tätig_bei",
"war", "beschäftigt_bei", "mitglied_von",
"war_eingeschult", "unterrichtet_in",

Familie

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

Gemeinsam nutzen ...



"war_eingeschult", "unterrichtet_in",

Familie
"verheiratet_mit", "kind_von", "eltern_von", "geschwister_von",

Religion & Kirche
"getauft_am", "beigetreten_am", "konfirmiert_am",
"hat_heiliges_abendmahl",

Interessen & Themen
"interessiert_sich_für", "beschäftigt_sich_mit", "forscht_zu",
"schreibt_über",

Reisen & Aufenthalte
"gereist_nach", "besucht", "angekommen_in", "abgereist_aus",
"aufgehalten_in", "zurückgekehrt_nach", "verließ", "war_in",
"war_am", "gefahren_nach"] class ExtractedInfo(BaseModel):
subjekt: str = Field(description="extrahierte Subjekt-Entität, z.B.
Daniel Wilhelm Suhl, ich, er, wir, meine I. Frau, seliger Bruder")
subjekt_type: str = Field(description="Typ der Subjekt-Entität, z.B.
person")
prädikat: str = Field(description="Beziehung zwischen Subjekt und
Objekt")
objekt: str = Field(description="extrahierte Objekt-Entität, z.B.
Gnadenthal, Aeltesten-Konferenz")
objekt_type: str = Field(description="Typ der Objekt-Entität, z.B.
location, organisation")
zeit: str = Field(default=None, description="Datumsangabe zum
Ereignis, z.B. 30 April 1858 oder leer, falls nicht genannt") # Parser
mit neuem Mode' ↓
parser = PydanticOutputParser(pydantic_object=ExtractedInfo) #

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

↑ Gemeinsam nutzen ...

parser = PydanticOutputParser(pydantic_object=ExtractedInfo) #
 Systemprompt
 system_prompt = PromptTemplate(
 template="""
 Du arbeitest mit biographischen, historischen und religiösen Texten
 der Herrnhuter Brüdergemeine.
 Deine Aufgabe ist es, Relationen zwischen Personen, Orten,
 Organisationen und Daten zu extrahieren.

 Gib ausschließlich eine JSON-Liste mit Objekten zurück. Jedes
 Objekt enthält:
 - "subjekt", "subjekt_type", "prädikat", "objekt", "objekt_type", "zeit".

 Die erlaubten Entitätstypen sind: {entity_types}
 Die erlaubten Beziehungstypen sind: {relation_types}

 KEINE Erklärungen oder zusätzlichen Texte!
 """,
 input_variables=["entity_types", "relation_types"]
)
 system_message_prompt =
 SystemMessagePromptTemplate(prompt=system_prompt) #
 Humanprompt
 human_prompt = PromptTemplate(
 template="""
 Hier ist ein Satz aus einem historischen Lebenslauf:
 "{text}"

 Folgende Entitäten sind im Satz enthalten:
 personen: {perso }
 orte: {orte}
 date: {date}
 """
)
 human_message_prompt =
 HumanMessagePromptTemplate(prompt=human_prompt) #
 Chatmodell + Prompt-Kette
 chat_prompt =
 ChatPromptTemplate.from_messages([system_message_prompt,
 human_message_prompt])
 model = ChatOllama(model="llama3", temperature=0.7)
 chain = LLMChain(llm=model, prompt=chat_prompt) results = []

 # Extraktion starten
 for entry in input_data:
 result = chain.run(entry)
 entity_types=entity_types

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

↑ Gemeinsam nutzen ...

Folgende Entitäten sind im Satz enthalten:
 personen: {personen}
 orte: {orte}
 date: {date}
 organisationen: {organisationen}

 Nutze nur die angegebenen Entitäten. Finde sinnvolle Relationen
 zwischen diesen Entitäten, z. B. wann und wo jemand geboren oder
 gestorben ist, wo jemand lebte oder tätig war.

 Wenn sinnvoll, gib zusätzlich im Feld "zeit" das relevante Datum an.

 {format_instructions}
 """,
 input_variables=["text", "personen", "orte", "date",
 "organisationen"],
 partial_variables={"format_instructions":
 parser.get_format_instructions()}
)
 human_message_prompt =
 HumanMessagePromptTemplate(prompt=human_prompt) #
 Chatmodell + Prompt-Kette
 chat_prompt =
 ChatPromptTemplate.from_messages([system_message_prompt,
 human_message_prompt])
 model = ChatOllama(model="llama3", temperature=0.7)
 chain = LLMChain(llm=model, prompt=chat_prompt) results = []

 # Extraktion starten
 for entry in input_data:
 result = chain.run(entry)
 entity_types=entity_types

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

[Gemeinsam nutzen](#) ...


```
# Extraktion starten
for entry in input_data:
    result = chain.run(
        entity_types=entity_types,
        relation_types=relation_types,
        text=entry["text"],
        personen=", ".join(entry.get("personen", [])),
        orte=", ".join(entry.get("orte", [])),
        date=", ".join(entry.get("date", [])),
        organisationen=", ".join(entry.get("organisationen", []))
    )
    # hier gleich transformieren, kein triple.json
    try:
        relations = json.loads(result)
    except json.JSONDecodeError:
        relations = []

    transformed = {
        "text": entry.get("text", ""),
        "ref": entry.get("ref", ""),
        "date": entry.get("date", ""),
        "autor": entry.get("autor", ""),
        "graph": relations
    }
    results.append(transformed)

# Speichere Ergebnisse
with open("data/5.4.2_RE/triple.json", "w", encoding="utf-8") as f:
    json.dump(results, f, ensure_ascii=False, indent=2)
```

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

[Gemeinsam nutzen](#) ...


Nachgedacht für 23s

Klar! Ich bin den Code einmal systematisch durchgegangen. Kurzfassung: **Es werden alle definierten Variablen/Imports verwendet.** Ich sehe keine offensichtlichen "toten" Variablen.

Check nach Bereichen

Imports

- `json` ✓
- `BaseModel`, `Field` (für `ExtractedInfo`) ✓
- `ChatOllama` ✓
- `PromptTemplate`, `SystemMessagePromptTemplate`, `HumanMessagePromptTemplate`, `ChatPromptTemplate` ✓
- `LLMChain` ✓
- `PydanticOutputParser` ✓

Top-Level Variablen / Konstanten

- `input_data` (für die Schleife) ✓
- `entity_types` (in `system_prompt` + `chain.run`) ✓
- `relation_types` (in `system_prompt` + `chain.run`) ✓

Klasse / Parser

- `ExtractedInfo` (als Schema für `PydanticOutputParser`) ✓
- `parser` (liefert `format_instructions` im `human_prompt`) ✓

Prompts / Chain

- `system_prompt` → in `system_message_prompt` ✓
- `system_message_prompt` → in `chat_prompt` ✓
- `human_prompt` → in `human_message_prompt` ✓

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

↑ Gemeinsam nutzen ...



- `human_message_prompt` → in `chat_prompt` ✓
- `chat_prompt` → in `LLMChain(prompt=...)` ✓
- `model (ChatOllama)` → in `LLMChain(llm=...)` ✓
- `chain` → in der Schleife `chain.run(...)` ✓

Laufzeit-Variablen in der Schleife

- `entry` ✓
- `result` (Roh-LLM-Antwort) ✓
- `relations` (geparstes JSON bzw. `[]`) ✓
- `transformed` (zusammengebautes Ergebnisobjekt) ✓
- `results` (Liste; wird am Ende gespeichert) ✓

Dateizugriffe

- `f` in beiden `with open(...)`-Blöcken ✓

Kleine Hinweise (keine Codeänderungen, nur Beobachtungen)

- In deinem Paste steht die schließende `]` von `relation_types` und `class ExtractedInfo` auf derselben Zeile (`... "gefahren_nach"] class ExtractedInfo(...):`). Das wäre syntaktisch ein Problem, falls es im echten File genauso ist. Vermutlich ist das nur ein Formatierungsfehler beim Kopieren.
- Der Feldname `prädikat` mit Umlaut ist okay, solange durchgängig so verwendet (ist es).
- Der Parametername `date` in `chain.run(...)` ist in Ordnung; es gibt keine Kollision mit einem Import. (Nur als Stilhinweis: manchmal nennt man das `daten / datenangaben`, um Verwechslungen mit `datetime.date` zu vermeiden.)
- Fehlerbehandlung: Bei `json.JSONDecodeError` setzt du `relations = []` – sinnvoll. Falls du später unterscheiden willst, warum es leer ist (wirklich keine Relationen vs. Parse-Fehler), wäre ein Logging hilfreich. (Kein Änderungswunsch, nur Anmerkung.)

Fazit: Aus Sicht „werden alle Variablen benutzt?“ – ja, alles wird verwendet. Keine ungenutzten

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

↑ Gemeinsam nutzen ...



Laufzeit-Variablen in der Schleife

- `entry` ✓
- `result` (Roh-LLM-Antwort) ✓
- `relations` (geparstes JSON bzw. `[]`) ✓
- `transformed` (zusammengebautes Ergebnisobjekt) ✓
- `results` (Liste; wird am Ende gespeichert) ✓

Dateizugriffe

- `f` in beiden `with open(...)`-Blöcken ✓

Kleine Hinweise (keine Codeänderungen, nur Beobachtungen)

- In deinem Paste steht die schließende `]` von `relation_types` und `class ExtractedInfo` auf derselben Zeile (`... "gefahren_nach"] class ExtractedInfo(...):`). Das wäre syntaktisch ein Problem, falls es im echten File genauso ist. Vermutlich ist das nur ein Formatierungsfehler beim Kopieren.
- Der Feldname `prädikat` mit Umlaut ist okay, solange durchgängig so verwendet (ist es).
- Der Parametername `date` in `chain.run(...)` ist in Ordnung; es gibt keine Kollision mit einem Import. (Nur als Stilhinweis: manchmal nennt man das `daten / datenangaben`, um Verwechslungen mit `datetime.date` zu vermeiden.)
- Fehlerbehandlung: Bei `json.JSONDecodeError` setzt du `relations = []` – sinnvoll. Falls du später unterscheiden willst, warum es leer ist (wirklich keine Relationen vs. Parse-Fehler), wäre ein Logging hilfreich. (Kein Änderungswunsch, nur Anmerkung.)

Fazit: Aus Sicht „werden alle Variablen benutzt?“ – ja, alles wird verwendet. Keine ungenutzten Variablen/Imports gefunden.



+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

5.4.3 Datenabgleich

The image displays two instances of the ChatGPT interface in a web browser. Both interfaces have a dark sidebar on the left with icons for home, search, and other functions.

Top Chat Window:

- Header: "ChatGPT 5" with a dropdown arrow and a link "Gemeinsam nutzen".
- Message: A German prompt asking to check if the provided code is logically structured and uses all variables, and to fix it if not.
- Code Block:

```
import json
import pandas as pd
import requests # Input-Datei laden
with open("data/5.4.2_RE/tripl_bereinig.json", 'r', encoding='utf-8')
as f:
    data = json.load(f) # DataFrame strukturieren
records = []
for entry in data:
    for g in entry.get("graph", []):
        records.append({
            "text": entry.get("text", ""),
            "ref": entry.get("ref", ""),
            "nbg": "",
            "datei": entry.get("datei", ""),
            "autor": entry.get("autor", ""),
            "subjekt": g.get("subjekt", ""),
            "subjekt_type": g.get("subjekt_type", ""),
            "q_subjekt": "",
            "prädikat": g.get("prädikat", ""),
            "p_wert": "",
            "objekt": g.get("objekt", ""),
            "objekt_type": g.get("objekt_type", ""),
            "q_objekt": "",
            "zeit": g.get("zeit", "")
        })

df = pd.DataFrame(records) #Q-ID (Entitäten)
```
- Input Field: "+ Stelle irgendeine Frage"
- Footer: "ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe Cookie-Voreinstellungen."

Bottom Chat Window:

- Header: "ChatGPT 5" with a dropdown arrow and a link "Gemeinsam nutzen".
- Code Block:

```
df = pd.DataFrame(records) #Q-ID (Entitäten)

def get_wikidata_qid(label):
    url = "https://www.wikidata.org/w/api.php"
    params = {
        "action": "wbsearchentities",
        "search": label,
        "language": "de",
        "format": "json",
        "type": "item",
        "limit": 1
    }
    resp = requests.get(url, params=params)
    data = resp.json()
    if data.get("search"):
        return data["search"][0]["id"]
    return ""

#P-ID (Properties)
def get_property_id(label):
    url = "https://www.wikidata.org/w/api.php"
    params = {
        "action": "wbsearchentities",
        "search": label,
        "language": "de",
        "format": "json",
        "type": "property",
        "limit": 1
    }
    resp = request.t(url, params=params)
    data = resp.json()
```
- Input Field: "+ Stelle irgendeine Frage"
- Footer: "ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe Cookie-Voreinstellungen."

The screenshot displays two separate chat interactions with ChatGPT. Each interaction shows a user prompt followed by a detailed Python script that interacts with Wikidata via SPARQL queries.

Top Interaction:

- User Prompt:** "Ich habe eine Liste von Subjekten und Prädikaten, die ich mit Wikidata verknüpfen möchte. Kannst du mir helfen, die entsprechenden Q-IDs zu finden?"
- ChatGPT Response:** A Python script that uses the `requests` library to query Wikidata. It defines functions to get Q-IDs for subjects and objects based on their labels. The script processes a dictionary of subjects and predicates, returning a mapping of Q-IDs.

Bottom Interaction:

- User Prompt:** "Kannst du mir zeigen, wie ich die Koordinaten einer Entität aus Wikidata abrufen kann?"
- ChatGPT Response:** A Python script that uses the `requests` library to query Wikidata for coordinates. It constructs a SPARQL query to retrieve the coordinates of a specific entity (QID) and returns them as a tuple of latitude and longitude.

ChatGPT 5 ▾

Gemeinsam nutzen ...



```
(objekt_type == "location")
df_weg = df[df["objekt_type"] == "location"].copy()

# Koordinaten abrufen
df_weg["lat"], df_weg["lon"] =
zip(*df_weg["q_objekt"].map(get_coordinates))

# Ergebnis zurück in Haupt-DataFrame schreiben
df.loc[df_weg.index, "lat"] = df_weg["lat"]
df.loc[df_weg.index, "lon"] = df_weg["lon"]
df.to_excel("data/5.4.2_RE/graphen.xlsx", index=False)
```

Nachgedacht für 16s

Klar – hier ist ein Review deines Codes (ohne Änderungen am Code selbst), mit Fokus auf Logik/Struktur und ob alle Variablen genutzt werden.

Struktur & Logik – Befunde

1. Syntax-/Zeilenumbrüche (hart kritisch)

- `df["p_wert"] = df["prädikat"].map(pid_map) def get_coordinates(qid):`
Zwischen dieser Zuweisung und der Funktionsdefinition fehlt ein Zeilenumbruch. So kompiliert das nicht.
- `df.loc[df_weg.index, "lon"] = df_weg["lon"] df.to_excel(...)`
Auch hier fehlt ein Zeilenumbruch zwischen der Zuweisung und `df.to_excel(...)`.

2. Potentieller Fehler bei leerem DataFrame

- `df_weg["lat"], df_weg["lon"] = zip(*df_weg["q_objekt"].map(get_coordinates))`
Wenn `df_weg` leer ist (keine `objekt_type == "location"`), wirft `zip(*[])` einen Fehler („not enough values to unpack“).
- Logisch wäre vorab zu prüfen, ob `len(df_weg) > 0`.

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

Gemeinsam nutzen ...



- Logisch wäre vorab zu prüfen, ob `len(df_weg) > 0`.
- 3. Rückgabestruktur `get_coordinates`
 - Die Funktion gibt immer ein Tupel `(lat, lon)` zurück (auch bei Fehlern `(None, None)`), was gut zur späteren Entpackung passt.
 - Parsing: Wikidata liefert `Point(lon lat)`. Du tauschst korrekt in `return float(lat), float(lon)`.
- 4. APIs & Robustheit
 - `get_wikidata_qid` / `get_property_id`: kein Timeout/Fehlerhandling. Bei Netzproblemen könnte das hängen oder Exceptions werfen.
 - Beide Funktionen rufen auch für leere Labels (z. B. `""`) die API auf, falls solche Strings in den Unique-Listen vorkommen. Das ist nicht falsch, aber unnötig.
- 5. Leistung/Rate-Limits
 - Für jede **einzigartige** Entität/Property wird ein Request geschickt – gut für Reduktion.
 - Kein Throttling/Backoff: bei vielen Unique-Werten könnten Wikidata-Limits greifen.
- 6. Spaltennamen/Encoding
 - Du nutzt Umlaute (`"prädikat"`). Das funktioniert in Pandas, aber kann in Excel-Tools/Downstream-Code gelegentlich unbequem sein. Inhaltlich ok.
- 7. SettingWithCopy
 - `df_weg = df[df["objekt_type"] == "location"].copy() - .copy()` ist korrekt, mindert SettingWithCopy -Risiken.
- 8. Ausgabeschritt
 - `df.to_excel(...)` setzt voraus, dass eine geeignete Engine (z. B. `openpyxl`) installiert ist. Sonst Exception. Logisch ok.
- 9. Kommentare & Lesbarkeit
 - Kommentar `# Input-Datei laden` steht hinter `import requests` auf derselben Zeile – verwirrend, aber funktional egal.
 - Kommentare strukturieren den Ablauf ordentlich.

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

Gemeinsam nutzen ...

- Kommentare strukturieren den Ablauf ordentlich.

Variablen-/Spaltennutzung

- **Imports:** `json`, `pandas` as `pd`, `requests` – alle verwendet.
- **Zwischenvariablen:** `records`, `df`, `unique_subjekte/objekte/prädikate`, `qid_map_*`, `pid_map`, `df_weg` – alle verwendet.
- **Funktionsparameter / Rückgaben:** genutzt wie vorgesehen.
- **Spalten:**
 - Erzeugt und weiter genutzt: `q_subjekt`, `q_objekt`, `p_wert`, `lat`, `lon`.
 - Erzeugt, aber nicht weiter ausgewertet: `nbg` (wird nur als leere Spalte angelegt). Das ist kein Fehler, aber wenn das Absicht ist, ist es okay; andernfalls wäre es „unbenutzt“ im Sinne der Logik.
- Fehlerobjekt `e` in `except` wird nur für Print genutzt – in Ordnung.

Konkrete Risiken/Zusammenfassung

- **Build-Stopper:** Zwei fehlende Zeilenumbrüche (siehe Punkt 1) verhindern die Ausführung.
- **Laufzeitfehler:** möglich bei leerem `df_weg` wegen `zip(*)` (Punkt 2).
- **Robustheit:** API-Requests ohne Timeout/Fehlerbehandlung bei `get_wikidata_qid / get_property_id` (Punkt 4).
- **Unbenutzte Spalte:** `nbg` (nur erstellt, sonst nicht genutzt). Wenn beabsichtigt, ok; ansonsten „tote“ Spalte.

Optionale Verbesserungs-Ideen (ohne Codeänderung jetzt)

- Zeilenumbrüche korrigieren (Pflicht).
- Vor `zip(*)` prüfen, ob `df_weg` nicht leer ist.
- Timeouts & rudimentäres Error-Handling in den Wikidata-Lookup-Funktionen hinzufügen.
- Leere Strings vor den Unique-Lookups filtern, um unnötige Requests zu vermeiden.
- (Nur wenn gewünscht) `nbg` entweder nutzen oder weglassen.

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...

- Leere Strings vor den Unique-Lookups filtern, um unnötige Requests zu vermeiden.
- (Nur wenn gewünscht) `nbg` entweder nutzen oder weglassen.

Wenn du möchtest, kann ich dir im nächsten Schritt eine bereinigte Version mit diesen minimalen Korrekturen zeigen (ohne die Logik zu verändern).

📄 📌 🗨️ 🔍 🔄 📤 📥

kannst du es bitte korrigieren, aber nur das

Nachgedacht für 8s

Alles klar — ich habe nur die harten Fehler korrigiert (fehlende Zeilenumbrüche) und den möglichen Crash bei leerem `df_weg` abgefangen. An der Logik/Struktur sonst nichts verändert.

```
python
import json
import pandas as pd
import requests # Input-Datei laden

with open("data/5.4.2_RE/tripl_bereinigt.json", 'r', encoding='utf-8') as f:
    data = json.load(f) # DataFrame strukturieren

records = []
for entry in data:
    for g in entry.get("graph", []):
        records.append({
            "text": entry.get("text", ""),
            "ref": entry.get("ref", "")
        })
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[Gemeinsam nutzen](#) ...


```
records = []
for entry in data:
    for g in entry.get("graph", []):
        records.append({
            "text": entry.get("text", ""),
            "ref": entry.get("ref", ""),
            "nbg": "",
            "datei": entry.get("datei", ""),
            "autor": entry.get("autor", ""),
            "subjekt": g.get("subjekt", ""),
            "subjekt_type": g.get("subjekt_type", ""),
            "q_subjekt": "",
            "prädikat": g.get("prädikat", ""),
            "p_wert": "",
            "objekt": g.get("objekt", ""),
            "objekt_type": g.get("objekt_type", ""),
            "q_objekt": "",
            "zeit": g.get("zeit", "")
        })

df = pd.DataFrame(records) # Q-ID (Entitäten)

def get_wikidata_qid(label):
    url = "https://www.wikidata.org/w/api.php"
    params = {
        "action": "wbsearchentities",
        "search": label,
        "language": "de",
        "format": "json",
        "type": "item",
        "limit": 1
    }
```

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[Gemeinsam nutzen](#) ...


```
"limit": 1
}
resp = requests.get(url, params=params)
data = resp.json()
if data.get("search"):
    return data["search"][0]["id"]
return ""

# P-ID (Properties)
def get_property_id(label):
    url = "https://www.wikidata.org/w/api.php"
    params = {
        "action": "wbsearchentities",
        "search": label,
        "language": "de",
        "format": "json",
        "type": "property",
        "limit": 1
    }
    resp = requests.get(url, params=params)
    data = resp.json()
    if data.get("search"):
        return data["search"][0]["id"]
    return ""

# Q-ID für Subjekte
unique_subjekte = df["subjekt"].unique()
qid_map_subjekt = {}
for subj in unique_subjekte:
    qid = get_wikidata_qid(subj)
    qid_map_subjekt[subj] = qid
```

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...



```
# Q-ID für Subjekte
unique_subjekte = df["subjekt"].unique()
qid_map_subjekt = {}
for subj in unique_subjekte:
    qid = get_wikidata_qid(subj)
    qid_map_subjekt[subj] = qid

df["q_subjekt"] = df["subjekt"].map(qid_map_subjekt)

# Q-ID für Objekte (achte auf den richtigen Spaltennamen: q_objekt)
unique_objekte = df["objekt"].unique()
qid_map_objekt = {}
for obj in unique_objekte:
    qid = get_wikidata_qid(obj)
    qid_map_objekt[obj] = qid

df["q_objekt"] = df["objekt"].map(qid_map_objekt)

# P-ID für Prädikate (nutze get_property_id!)
unique_prädikate = df["prädikat"].unique()
pid_map = {}
for prd in unique_prädikate:
    pid = get_property_id(prd)
    pid_map[prd] = pid

df["p_wert"] = df["prädikat"].map(pid_map)

def get_coordinates(qid):
    if not qid or not isinstance(qid, str) or not qid.startswith("Q"):
        return None, None

    url = "https://query.wikidata.org/sparql"
    query = """
SELECT ?coord WHERE {{
    wd:{qid} wdt:P625 ?coord.
}}
"""
    headers = {
        "Accept": "application/sparql-results+json",
        "User-Agent": "GeoKoordinatenBot/1.0 (example@domain.com)" # Eigene Adresse od
    }

    try:
        response = requests.get(url, params={"query": query}, headers=headers, timeout=
        response.raise_for_status()

        data = response.json()
        bindings = data.get("results", {}).get("bindings", [])
        if bindings:
            coord_str = bindings[0]["coord"]["value"]
            lon, lat = coord_str.replace("Point(", "").replace(")", "").split()
            return float(lat), float(lon)
        except Exception as e:
            print(f"Fehler bei {qid}: {e}")
            return None, None

# Optional: Nur für geografische Entitäten (objekt_type == "location")
df_weg = df[df["objekt_type"] == "location"].copy()
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...



```
def get_coordinates(qid):
    if not qid or not isinstance(qid, str) or not qid.startswith("Q"):
        return None, None

    url = "https://query.wikidata.org/sparql"
    query = """
SELECT ?coord WHERE {{
    wd:{qid} wdt:P625 ?coord.
}}
"""
    headers = {
        "Accept": "application/sparql-results+json",
        "User-Agent": "GeoKoordinatenBot/1.0 (example@domain.com)" # Eigene Adresse od
    }

    try:
        response = requests.get(url, params={"query": query}, headers=headers, timeout=
        response.raise_for_status()

        data = response.json()
        bindings = data.get("results", {}).get("bindings", [])
        if bindings:
            coord_str = bindings[0]["coord"]["value"]
            lon, lat = coord_str.replace("Point(", "").replace(")", "").split()
            return float(lat), float(lon)
        except Exception as e:
            print(f"Fehler bei {qid}: {e}")
            return None, None

# Optional: Nur für geografische Entitäten (objekt_type == "location")
df_weg = df[df["objekt_type"] == "location"].copy()
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

[Gemeinsam nutzen](#) ...

```
data = response.json()
bindings = data.get("results", {}).get("bindings", [])
if bindings:
    coord_str = bindings[0]["coord"]["value"]
    lon, lat = coord_str.replace("Point(", "").replace(")", "").split()
    return float(lat), float(lon)
except Exception as e:
    print(f"Fehler bei {qid}: {e}")
return None, None

# Optional: Nur für geografische Entitäten (objekt_type == "location")
df_weg = df[df["objekt_type"] == "location"].copy()

# Koordinaten abrufen (nur wenn Zeilen vorhanden sind)
if not df_weg.empty:
    df_weg["lat"], df_weg["lon"] = zip(*df_weg["q_objekt"].map(get_coordinates))

# Ergebnis zurück in Haupt-DataFrame schreiben
df.loc[df_weg.index, "lat"] = df_weg["lat"]
df.loc[df_weg.index, "lon"] = df_weg["lon"]

df.to_excel("data/5.4.2_RE/graphen.xlsx", index=False)
```

Wenn du möchtest, kann ich dir jetzt noch eine Version mit Timeouts/Fehlerbehandlung für die Wikidata-Suche oder mit kleinen Performance-Optimierungen machen—aber das habe ich bewusst nicht angefasst.



+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

[Gemeinsam nutzen](#) ...

```
was ist hier falsch: import pandas as pd
import requests

# Excel-Datei laden
df_weg = pd.read_excel("data/5.4.3_EL/graphen_bereinigt.xlsx")

# Funktion zum Abrufen der Koordinaten aus Wikidata
def get_coordinates(qid):
    if not qid or not isinstance(qid, str) or not qid.startswith("Q"):
        return None, None

    url = "https://query.wikidata.org/sparql"
    query = """
    SELECT ?coord WHERE {{
      wd:{qid} wdt:P625 ?coord.
    }}
    """
    headers = {
        "Accept": "application/sparql-results+json",
        "User-Agent": "GeoKoordinatenBot/1.0 (example@domain.com)"
    }

    try:
        response = requests.get(url, params={"query": query},
                                headers=headers, timeout=10)
        response.raise_for_status()

        data = response.json()
        bindings = data.get("results", {}).get("bindings", [])
        if bindings:
            coord_str = bindings[0]["coord"]["value"]
```

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

Gemeinsam nutzen ...



```

coord_str = bindings[0]["coord"]["value"]
lon, lat = coord_str.replace("Point(", "").replace(")", "").split()
return float(lat), float(lon)
except Exception as e:
    print(f"Fehler bei {qid}: {e}")
    return None, None

# Nur Zeilen mit objekt_type == "location"
df_locations = df_weg[df_weg["objekt_type"] == "location"].copy()

# Koordinaten abrufen
koordinaten = df_locations["q_objekt"].map(get_coordinates)
df_locations["lat"], df_locations["lon"] = zip(*koordinaten)

# Zurückschreiben in df_weg
df_weg.loc[df_locations.index, "lat"] = df_locations["lat"]
df_weg.loc[df_locations.index, "lon"] = df_locations["lon"]

```

Dein Code sieht grundsätzlich sauber aus – du machst alles richtig (Excel laden, Wikidata abfragen, Koordinaten parsen, in DataFrame zurückschreiben).
Ein paar Punkte, die Fehler verursachen können:

1. Reihenfolge von lat und lon

Wikidata gibt Koordinaten im WKT-Format "Point(LONG LAT)" zurück.

Dein Code:

```
python
+ Stelle irgendeine Frage
```

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5

Gemeinsam nutzen ...



Dein Code:

```
python
lon, lat = coord_str.replace("Point(", "").replace(")", "").split()
return float(lat), float(lon)
```

Hier vertauschst du bewusst lat und lon – aber weiter unten speicherst du:

```
python
df_locations["lat"], df_locations["lon"] = zip(*koordinaten)
```

Ergebnis: Lat und Lon sind vertauscht.

Richtig wäre entweder:

```
python
lon, lat = coord_str.replace("Point(", "").replace(")", "").split()
return float(lon), float(lat) # lon zuerst, lat danach
```

und später beim Zuweisen:

```
python
df_locations["lon"], df_locations["lat"] = zip(*koordinaten)
```

Oder du lässt lat, lon drin und passt nur die Rückgabe an:

```
python
return float(lat), float(lon) # aber das lässt die Zuweisung
```

```
python
+ Stelle irgendeine Frage
```

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

Gemeinsam nutzen ...

2. Fehler bei `zip(*koordinaten)`

Wenn **alle Abfragen fehlschlagen** und du nur `(None, None)` zurückbekommst, ist `koordinaten` eine Serie voller `(None, None)` – das geht.
Aber wenn `df_locations` **leer** ist (keine Zeilen mit "location"), dann kracht `zip(*koordinaten)` mit `ValueError: not enough values to unpack (expected 2, got 0)`.

Abhilfe: Vorher prüfen:

```
python
if not df_locations.empty:
    koordinaten = df_locations["q_objekt"].map(get_coordinates)
    df_locations["lat"], df_locations["lon"] = zip(*koordinaten)
    df_weg.loc[df_locations.index, ["lat", "lon"]] = df_locations[["lat", "lon"]]
```

3. Typprüfung auf `qid`

Du brichst ab, wenn `qid` nicht mit "Q" anfängt. Manche Wikidata-IDs sind aber nicht nur einfache `Q123`-Formate, sondern können in der Excel-Datei evtl. als Zahl gespeichert sein. Dann kommt `int` rein → Abbruch.

Das wäre abfangbar:

```
python
qid = str(qid)
if not qid.startswith("Q"):
    return None, None
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

Gemeinsam nutzen ...

4. Performance

Du schickst **für jede Zeile** eine eigene SPARQL-Query → sehr langsam. Wikidata drosselt dann wahrscheinlich. Besser wäre, alle QIDs gesammelt in einer SPARQL-Query abzufragen.

Ich tippe stark darauf, dass dein Hauptproblem die **vertauschten lat/lon** und/oder der Fehler bei **leerem DataFrame** ist.

Soll ich dir eine korrigierte, robuste Version des Codes schreiben, die auch mit leerem `df_locations` funktioniert und `lat/lon` garantiert richtig zuordnet?

Stelle irgendeine Frage

```
{
  "name": "TypeError",
  "message": "'NoneType' object is not iterable",
  "stack": "-----
TypeError                                Traceback (most recent call last)
Cell In[6], line 42
    40 # Koordinaten abrufen
    41 koordinaten = df_locations["q_objekt"].map(get_coordinates)
--> 42 df_locations["lat"], df_locations["lon"] = zip(*koordinaten)
    44 # Zurückschreiben in df_weg
    45 df_weg.loc[df_locations.index, ["lat", "lon"]] = df_locations["lat"]

TypeError: 'NoneType' object is not iterable"
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).





ChatGPT 5 ▾

[↑ Gemeinsam nutzen](#) ...

Nachgedacht für 17s

Der Fehler kommt daher, dass **mindestens ein Aufruf von `get_coordinates` `None` (statt `(None, None)`) zurückgibt**. Dann macht `zip(*koordinaten)` ein `zip(None, ...)` → `TypeError: 'NoneType' object is not iterable`.

Warum passiert das? In deiner Funktion fehlt am Ende ein `return (None, None)`, falls **keine Treffer** (`bindings` leer) zurückkommen. Dann fällt Python aus der Funktion ohne Rückgabe heraus → implizit `None`.

Hier ist eine robuste, korrigierte Version (inkl. richtiger Lat/Lon-Zuordnung aus `"Point(LON LAT)"` und ohne `zip(*...)`-Fallstrick):

```
python
import pandas as pd
import requests

# Excel-Datei laden
df_weg = pd.read_excel("data/5.4.3_EL/graphen_bereinigt.xlsx")

def get_coordinates(qid):
    # Immer String, damit z.B. numerische Zellen nicht scheitern
    if qid is None or (isinstance(qid, float) and pd.isna(qid)):
        return (None, None)
    qid = str(qid)
    if not qid.startswith("Q"):
        return (None, None)

    url = "https://query.wikidata.org/sparql"
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

ChatGPT 5 ▾

[↑ Gemeinsam nutzen](#) ...

```
url = "https://query.wikidata.org/sparql"
query = """
SELECT ?coord WHERE {{
  wd:{qid} wdt:P625 ?coord.
}}
"""
headers = {
    "Accept": "application/sparql-results+json",
    "User-Agent": "GeoKoordinatenBot/1.0 (example@domain.com)"
}

try:
    r = requests.get(url, params={"query": query}, headers=headers, timeout=10)
    r.raise_for_status()
    data = r.json()
    bindings = data.get("results", {}).get("bindings", [])
    if bindings:
        coord_str = bindings[0]["coord"]["value"] # "Point(LON LAT)"
        lon_str, lat_str = coord_str.replace("Point(", "").replace(")", "").split()
        # Lat/Lon korrekt zuordnen
        return (float(lat_str), float(lon_str))
    except Exception as e:
        print(f"Fehler bei {qid}: {e}")

# WICHTIG: Immer ein 2er-Tupel zurückgeben
return (None, None)

# Nur Zeilen mit objekt_type == "location"
df_locations = df_weg[df_weg["objekt_type"] == "location"].copy()

if not df_locations.empty:
    # Koordinaten schreiben
```

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[Gemeinsam nutzen](#) ...

```

print(f"Fehler bei {qid}: {e}")

# WICHTIG: Immer ein 2er-Tupel zurückgeben
return (None, None)

# Nur Zeilen mit objekt_type == "location"
df_locations = df_weg[df_weg["objekt_type"] == "location"].copy()

if not df_locations.empty:
    # Koordinaten abrufen
    coords = df_locations["q_objekt"].map(get_coordinates)
    # Sicherstellen, dass jedes Element ein 2er-Tupel ist
    coords = coords.apply(lambda x: x if isinstance(x, tuple) and len(x) == 2 else (None, None))
    # Ohne zip(...) - direkt in zwei Spalten expandieren
    df_locations[["lat", "lon"]] = pd.DataFrame(coords.tolist(), index=df_locations.index)

# Zurückschreiben in df_weg
df_weg.loc[df_locations.index, ["lat", "lon"]] = df_locations[["lat", "lon"]]

```

Kurz:

- Ursache: fehlendes `return (None, None)` bei leeren `bindings`.
- Fix: Immer ein 2-Tupel zurückgeben und beim Aufteilen nicht `zip(...)`, sondern `DataFrame(coords.tolist())` verwenden.
- Bonus: Lat/Lon korrekt aus `"Point(LON LAT)"` extrahiert.



+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).

6. SRAPQL



ChatGPT 5 ▾

Gemeinsam nutzen ...



ich habe den folgenden Code: #defaultView:Graph
 SELECT ?person ?rgb ?personLabel ?labelDE ?prop ?
 propLabel ?value ?valueLabel ?start ?end ?pointInTime ?
 source WHERE { # Grundmenge: Personen auf der
 Arbeitsliste ?person wdt:P31 wd:Q5 ; wdt:P5008
 wd:Q136079887 . OPTIONAL { ?person rdfs:label ?
 labelDE . FILTER(LANG(?labelDE) = "de") } # Einheitliche
 Property-Liste VALUES ?prop { wd:P569 wd:P570 #
 Geburts-/Sterbedatum wd:P19 wd:P20 wd:P551 #
 Geburtsort / Sterbeort / Wohnsitz wd:P22 wd:P25
 wd:P3373 # Vater / Mutter / Geschwister wd:P26 wd:P40
 wd:P1971 # Ehepartner / Kinder / Anzahl Kinder wd:P106
 wd:P463 wd:P1066 # Beruf / Mitglied von / Schüler von
 wd:P69 # Ausgebildet an wd:P937 # Wirkungsort
 wd:P793 wd:P1344 # Bedeutendes Ereignis / Teilnehmer
 an } # Prädikate ableiten ?prop wikibase:directClaim ?wdt
 ; wikibase:claim ?p ; wikibase:statementProperty ?ps . { ?
 person ?wdt ?value . } UNION { ?person ?p ?st . ?st ?ps ?
 value . OPTIONAL { ?st pq:P580 ?start } OPTIONAL { ?st
 pq:P582 ?end } OPTIONAL { ?st pq:P585 ?pointInTime }
 OPTIONAL { ?st prov:wasDerivedFrom ?ref . OPTIONAL {
 ?ref pr:P854 ?source } } } # Farbegelung BIND(IF(?prop
 IN (wd:P22, wd:P25, wd:P3373), "800080", # Eltern &
 Geschwister → Lila IF(?prop = wd:P26, "E91E63", #
 Ehepartner → Pink IF(?prop = wd:P40, "FF8C00", #
 Kinder → Orange IF(?prop = wd:P1971, "FFC107", #
 Anzahl Kinder → Amber IF(?prop = wd:P937, "FF0000", #
 Wirkungsort → Rot IF(?prop IN (wd:P19, wd:P20,
 wd:P551), "2CA02C", # Orte (Geburts-/Sterbeort,
 Wohnsitz) → Grün IF(?prop IN (wd:P106, wd:P463,
 wd:P1066), "FFD700", #
 Tätigkeiten/Zugehörigkeit/Schüler von → Gelb IF(?prop =
 wd:P69, "17A2B8", # Ausbildung → Türkis IF(?prop =
 wd:P793, "6D4C41", # Bedeutendes Ereignis → Braun
 IF(?prop = wd:P1344, "00BCD4", # Teilnehmer an →
 Cyan IF(?prop IN (wd:P569, wd:P570), "7F8C8D", #
 Geburts-/Sterbedatum → Grau "1F77B4" # Standard →
 Blau)))))))) AS ?rgb) SERVICE wikibase:label {
 bd:serviceParam wikibase:language "mul,en,de". } }
 ORDER BY ?personLabel ?propLabel ?valueLabel ?start
 Problem ist, dass die Farben jedes mal "springen", wenn
 ich die Abfrage laufen lasse. Woran kann das liegen?



Nachgedacht für 20s

+ Stelle irgendeine Frage

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

Gemeinsam nutzen ...



ich die Abfrage laufen lasse. Woran kann das liegen?

Nachgedacht für 20s

Kurzfassung: Das liegt nicht an deinem `BIND`, sondern an der **Graph-Ansicht vom WDQS**. Die Graph-Visualisierung vergibt für Kategorien (z. B. `?prop / ?propLabel`) bei jedem Rendern **neue Palettenzuordnungen** – und ignoriert außerdem die Reihenfolge deiner Ergebniszeilen. Dadurch "springen" die Farben zwischen den Läufen.

So machst du die Farben stabil:

1. **Explizite Farbsäule verwenden und in der UI auswählen**
 - Du hast schon `?rgb` gebunden.
 - In der Graph-Ansicht oben rechts im Dropdown „Color by“ (bzw. „Farbe“) „`rgb`“ auswählen. Dann nimmt die Visualisierung exakt deine Hexwerte und ordnet sie nicht mehr jedes Mal neu zu.
2. (Optional) **Kleine Robustheits-Tweaks**
 - Nutze nur **Hex ohne #** (so wie du es schon tust), das erwartet die WDQS-Graph-Ansicht.
 - Die Sortierung am Ende (`ORDER BY ...`) hilft hier nicht, weil die Graph-Ansicht die Ergebnisreihenfolge ignoriert.
3. Falls du **Knoten** (statt Kanten) einfärben willst:
 - Die Graph-Ansicht kann nur eine Spalte zur Farbgebung heranziehen. Dann musst du diese Farbe an die Knoten-„Zeilen“ binden (also dort, wo `?person` bzw. der jeweilige Knoten im Resultset auftaucht) und im Dropdown ebenfalls deine Farbspalte wählen.

Warum es "springt":

- Das Resultset kommt durch `UNION / OPTIONAL` bei jedem Lauf in leicht anderer Reihenfolge.
- Die Graph-Ansicht mappt Kategorien → Farben **dynamisch** (je nach zuerst gesehenen Kategorien). Ohne explizite `?rgb`-Auswahl ist das jedes Mal anders.

Fazit: In deiner Abfrage passt die Farblogik. Wähle in der Graph-Ansicht „Color by: rgb“ – dann bleiben die Farben konstant.

kannst du dieses Problem lösen?



+ Stelle irgendeine Frage



SV

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

Gemeinsam nutzen ...



Nachgedacht für 5s

alles klar — hier sind **zwei komplette, lauffähige Varianten** für den Graph-Viewer.

Beide sind „stabil“, d. h. keine zufälligen Weiß/Blau-Sprünge mehr.

Variante A — Stabile Knotenfarben (Personen blau, Ziel-/Wertknoten farbig je Property)

Beibehaltung deiner großen Ergebnisliste (mit Zeiten, Quellen etc.). Farben werden **nur** für Item-Knoten gesetzt; Datum/Zahl bleiben neutral (Graph-Viewer-Limit).

sparql

Code kopieren

```
#defaultView:Graph
SELECT
  ?person ?rgb ?personLabel
  ?labelDE
  ?prop ?propLabel
  ?value ?valueLabel
  ?start ?end ?pointInTime ?source
WHERE {
  ### Grundmenge Personen
  ?person wdt:P31 wd:Q5 ;
    wdt:P5008 wd:Q136079887 .
  OPTIONAL { ?person rdfs:label ?labelDE . FILTER(LANG(?labelDE) = "de")

  ### Einheitliche Property-Liste
  VALUES ?prop {
    wd:P569 wd:P570                # Geburts-/Sterbedatum
    wd:P19 wd:P20 wd:P551          # Geburtsort / Sterbeort / Wohnsitz
    wd:P22 wd:P25 wd:P3373         # Vater / Mutter / Geschwister
    wd:P26 wd:P40 wd:P1971         # Ehepartner / Kinder / Anzahl Kinde
    wd:P106 wd:P463 wd:P1066       # Beruf / Mitglied von / Schüler von
    wd:P69                          # Ausgebildet an
    wd:P937                         # Wirkungsort
    wd:P793 wd:P1344               # Bedeutendes Ereignis / Teilnehmer
  }

  ### Prädikate ableiten
  ?prop wikibase:directClaim ?wdt ;
    wikibase:claim ?p ;
    wikibase:statementProperty ?ps .

  ### Kanten erzeugen (OHNE ?rgb!)
  {
```

+ Stelle irgendeine Frage



SV

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[Gemeinsam nutzen](#) ...


Code kopieren

```
{
  { ?person ?wdt ?value . }
  UNION
  {
    ?person ?p ?st .
    ?st ?ps ?value .
    OPTIONAL { ?st pq:P580 ?start }
    OPTIONAL { ?st pq:P582 ?end }
    OPTIONAL { ?st pq:P585 ?pointInTime }
    OPTIONAL {
      ?st prov:wasDerivedFrom ?ref .
      OPTIONAL { ?ref pr:P854 ?source }
    }
  }
}

### Farben stabil zuweisen (separate UNION-Zweige)

# 1) Personen-Knoten immer Blau
UNION {
  ?person wdt:P31 wd:Q5 ;
    wdt:P5008 wd:Q136079887 .
  BIND("1F77B4" AS ?rgb) # Blau
}

# 2) Wert-Knoten farbig je Property (nur für Items)
UNION {
  ?person wdt:P31 wd:Q5 ;
    wdt:P5008 wd:Q136079887 .
  ?prop wikibase:directClaim ?wdt ;
    wikibase:claim ?p ;
    wikibase:statementProperty ?ps .
  {
    ?person ?wdt ?value .
  }
  UNION
  {
    ?person ?p ?st .
    ?st ?ps ?value .
  }
  FILTER(isIRI(?value)) # nur Item-Knoten bekommen Farbe

  BIND(
    IF(?prop IN (wd:P22, wd:P25, wd:P3373), "800080", # Fami
    IF(?prop = wd:P26, "E91E63", # Ehep
    IF(?prop = wd:P40, "FF8C00", # Kind
```

+ Stelle irgendeine Frage



SV

 ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

[↑ Gemeinsam nutzen](#) ...


Code kopieren

```

### Kanten erzeugen (OHNE ?rgb!)
{
  { ?person ?wdt ?value . }
  UNION
  {
    ?person ?p ?st .
    ?st ?ps ?value .
    OPTIONAL { ?st pq:P580 ?start }
    OPTIONAL { ?st pq:P582 ?end }
    OPTIONAL { ?st pq:P585 ?pointInTime }
    OPTIONAL {
      ?st prov:wasDerivedFrom ?ref .
      OPTIONAL { ?ref pr:P854 ?source }
    }
  }
}

### Farben stabil zuweisen (separate UNION-Zweige)

# 1) Personen-Knoten immer Blau
UNION {
  ?person wdt:P31 wd:Q5 ;
    wdt:P5008 wd:Q136079887 .
  BIND("1F77B4" AS ?rgb) # Blau
}

# 2) Wert-Knoten farbig je Property (nur für Items)
UNION {
  ?person wdt:P31 wd:Q5 ;
    wdt:P5008 wd:Q136079887 .
  ?prop wikibase:directClaim ?wdt ;
    wikibase:claim ?p ;
    wikibase:statementProperty ?ps .
  {
    ?person ?wdt ?value .
  }
  UNION
  {
    ?person ?p ?st .
    ?st ?ps ?value .
  }
  FILTER(isIRI(?value)) # nur Item-Knoten bekommen Farbe

  BIND(
    IF(?prop IN (wd:P22, wd:P25, wd:P3373), "800080",
    # Fami

```

+ Stelle irgendeine Frage



SV

 ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[↑ Gemeinsam nutzen](#) ...


```

BIND(
  IF(?prop IN (wd:P22, wd:P25, wd:P3373), "800080",      # Fami
  IF(?prop = wd:P26,                                     # Ehep
  IF(?prop = wd:P40,                                     # Kind
  IF(?prop = wd:P1971,                                   # Anza
  IF(?prop = wd:P937,                                    # Wirk
  IF(?prop IN (wd:P19, wd:P20, wd:P551),                 # Orte
  IF(?prop IN (wd:P106, wd:P463, wd:P1066),              # Täti
  IF(?prop = wd:P69,                                     # Ausb
  IF(?prop = wd:P793,                                    # Bede
  IF(?prop = wd:P1344,                                   # Teil
  IF(?prop IN (wd:P569, wd:P570),                         # Gebu
                                     "1F77B4"            # Fall

  ))))))) AS ?rgb
)
}

SERVICE wikibase:label { bd:serviceParam wikibase:language "mul,de,en"
}

ORDER BY ?personLabel ?propLabel ?valueLabel ?start

```

Code kopieren

Variante B — Kanten einfärben (sehr stabil; Knoten bleiben neutral/gleich)

Wenn dir wichtig ist, dass **jede Beziehungslinie** die richtige Farbe hat (und nicht die Knoten): hier eine schlanke, robuste Kanten-Ansicht.

```

sparql

#defaultView:Graph
SELECT
  ?person ?personLabel
  ?prop ?propLabel
  ?value ?valueLabel
  ?rgb
WHERE {
  ### Grundmenge Personen
  ?person wdt:P31 wd:Q5 ;
          wdt:P5008 wd:Q136079887 .

  ### Property→Farbe Mapping (stabil, deterministisch)
  VALUES (?prop ?rgb) {
    (wd:P22 "800080") # Vater → Lila
    (wd:P25 "800080") # Mutter → Lila

```

Code kopieren

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[↑ Gemeinsam nutzen](#) ...


```
(wd:P40 "FF8C00") # Kinder → Orange
(wd:P1971 "FFC107") # Anzahl Kinder → Amber
(wd:P937 "FF0000") # Wirkungsort → Rot
(wd:P19 "2CA02C") # Geburtsort → Grün
(wd:P20 "2CA02C") # Sterbeort → Grün
(wd:P551 "2CA02C") # Wohnsitz → Grün
(wd:P106 "FFD700") # Beruf → Gelb
(wd:P463 "FFD700") # Mitglied von → Gelb
(wd:P1066 "FFD700") # Schüler/Assistent von → Gelb
(wd:P69 "17A2B8") # Ausgebildet an → Türkis
(wd:P793 "6D4C41") # Bedeutes Ereignis → Braun
(wd:P1344 "00BCD4") # Teilnehmer an → Cyan
(wd:P569 "7F8C8D") # Geburtsdatum → Grau
(wd:P570 "7F8C8D") # Sterbedatum → Grau
}

### Direktwert oder Statementwert (vereinheitlicht)
?prop wikibase:directClaim ?wdt ;
    wikibase:claim ?p ;
    wikibase:statementProperty ?ps .

{
    ?person ?wdt ?value .
}
UNION
{
    ?person ?p ?st .
    ?st ?ps ?value .
}

SERVICE wikibase:label { bd:serviceParam wikibase:language "mul,de,en"
}
ORDER BY ?personLabel ?propLabel ?valueLabel
```

[Code kopieren](#)

Wenn du mir sagst, **welche Variante** du nutzen möchtest (A = Knotenfarben, B = Kantenfarben), passe ich dir den Code gern weiter an (z. B. zusätzliche Properties, andere Farbtöne, Edge-Dicke, Tooltip-Felder).

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

[↑ Gemeinsam nutzen](#) ...


kannst du bitte die Abfrage prüfen: #defaultView:Graph

SELECT

?tätigkeit ?tätigkeitLabel

?edgeLabel

?date ?refURL ?person ?personLabel

WHERE {

?route wdt:P31 wd:Q5 ;

wdt:P5008 wd:Q136079887 .

?tätigkeit ?p ?stmt .

VALUES (?p ?ps ?role) {

(wdt:P106 "Tätigkeit")

(wdt:P69 "Ausbildung")

}

?stmt ?ps ?place .

OPTIONAL { ?stmt pq:P585 ?date . }

OPTIONAL { ?stmt prov:wasDerivedFrom/pr:P854 ?
refURL . }

OPTIONAL { ?place wdt:P625 ?coord . }

OPTIONAL {

?person wdt:P1344 ?tätigkeit .

?person wdt:P31 wd:Q5 .

}

BIND(?role AS ?edgeLabel)

BIND(

IF(?role = "Tätigkeit", "FF4136",

IF(?role = "Ausbildung", "0074D9",

)AS ?rgb

)

SERVICE wikibase:label { bd:serviceParam
wikibase:language "mul,de,en". }

}

ORDER BY ?tätigkeitLabel ?role

ich möchte Graph von Tätigkeit und Ausbildung von
Personen bild. ↓ es soll innerhalb wdt:P31 wd:Q5 und
wdt:P5008 wd:Q136079887 sein

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

[↑ Gemeinsam nutzen](#) ...


ich möchte Graph von Tätigkeit und Ausbildung von Personen bilden. es soll innerhalb wdt:P31 wd:Q5 und wdt:P5008 wd:Q136079887 sein

Nachgedacht für 12s

Klar – deine Query hat ein paar typische Wikidata-SPARQL-Fallstricke:

- Für Aussagen/Statements musst du **p:** und **ps:** (bzw. **pq:** für Qualifier) benutzen – nicht direkt **wdt:** an einem Statementknoten.
- In **VALUES** hast du Strings statt Properties gebunden.
- **?route** wird nie verwendet; gemeint ist wohl **?person**.
- Das **IF(...)** AS **?rgb** war syntaktisch kaputt.
- Für Zeitangaben sind bei P106/P69 i. d. R. **pq:P580** / **pq:P582** (Start/Ende) üblich; **pq:P585** (Zeitpunkt) gibt's seltener.

Hier ist eine bereinigte Version, die genau deine Anforderungen erfüllt und im **Graph-**View farbige Kanten (Tätigkeit rot, Ausbildung blau) zeigt:

sparql

[Code kopieren](#)

```
#defaultView:Graph
SELECT
  ?tätigkeit ?tätigkeitLabel
  ?edgeLabel
  ?date ?refURL ?person ?personLabel ?rgb
WHERE {
  # Personen, die auf der Fokusliste Q136079887 stehen
  ?person wdt:P31 wd:Q5 ;
    wdt:P5008 wd:Q136079887 .

  {
    # Tätigkeit (Beruf)
    ?person p:P106 ?stmt1 .
    ?stmt1 ps:P106 ?tätigkeit .
    OPTIONAL { ?stmt1 pq:P580 ?start1 . }
    OPTIONAL { ?stmt1 pq:P582 ?end1 . }
    OPTIONAL { ?stmt1 pq:P585 ?point1 . } # falls als Zeitpunkt mode
    OPTIONAL { ?stmt1 prov:wasDerivedFrom/pr:P854 ?refURL . }
    BIND("Tätigkeit" AS ?edgeLabel)
    BIND(COALESCE(?point1, ?start1, ?end1) AS ?date)
  }
  UNION
  {
    # Ausbildung (Bildungseinrichtung)
    ?person p:P107 ?stmt2 .
    ?stmt2 ps:P107 ?ausbildung .
    OPTIONAL { ?stmt2 pq:P580 ?start2 . }
    OPTIONAL { ?stmt2 pq:P582 ?end2 . }
    OPTIONAL { ?stmt2 pq:P585 ?point2 . }
    OPTIONAL { ?stmt2 prov:wasDerivedFrom/pr:P854 ?refURL . }
    BIND("Ausbildung" AS ?edgeLabel)
    BIND(COALESCE(?point2, ?start2, ?end2) AS ?date)
  }
}
```

+ Stelle irgendeine Frage



SV

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5

Gemeinsam nutzen ...

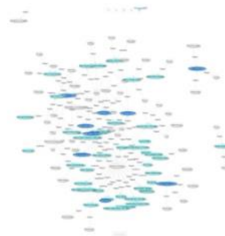


```
UNION
{
  # Ausbildung (Bildungseinrichtung)
  ?person p:P69 ?stmt2 .
  ?stmt2 ps:P69 ?tätigkeit .          # für den Graph vereinheit
  OPTIONAL { ?stmt2 pq:P580 ?start2 . }
  OPTIONAL { ?stmt2 pq:P582 ?end2 . }
  OPTIONAL { ?stmt2 pq:P585 ?point2 . }
  OPTIONAL { ?stmt2 prov:wasDerivedFrom/pr:P854 ?refURL . }
  BIND("Ausbildung" AS ?edgeLabel)
  BIND(COALESCE(?point2, ?start2, ?end2) AS ?date)
}

# Kantenfarbe im Graph-View
BIND(
  IF(?edgeLabel = "Tätigkeit", "FF4136",
    IF(?edgeLabel = "Ausbildung", "0074D9", "AAAAAA")
  ) AS ?rgb
)

SERVICE wikibase:label { bd:serviceParam wikibase:language "mul,de,en"
}
ORDER BY ?personLabel ?edgeLabel ?tätigkeitLabel
```

Wenn du lieber Start- und Enddatum **getrennt** sehen willst (statt einer zusammengefassten `?date`-Spalte), sag Bescheid – ich gebe dir dann eine Variante mit `?start / ?end`.



problem momentan, dass die menschen markiert wurden und nicht eigentliche Tätigkeiten oder Ausbildung:

```
#defaultView:Graph
SELECT
  ?tätigkeit ?tätigkeitLabel
```

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).



ChatGPT 5 ▾

Gemeinsam nutzen ...



```
?tätigkeit ?tätigkeitLabel
?place ?rgb ?placeLabel
?edgeLabel
?date ?refURL ?person ?personLabel
WHERE {
  ?route wdt:P31 wd:Q1322323 ;
    wdt:P5008 wd:Q136079887 .

  ?route ?p ?stmt .
  VALUES (?p ?ps ?role) {
    (wdt:P106 "Tätigkeit")
    (wdt:P69 "Ausbildung")
  }
  ?stmt ?ps ?place .

  OPTIONAL { ?stmt pq:P585 ?date . }
  OPTIONAL { ?stmt prov:wasDerivedFrom/pr:P854 ?
refURL . }
  OPTIONAL { ?place wdt:P625 ?coord . }

  OPTIONAL {
    ?person wdt:P1344 ?tätigkeit .
    ?person wdt:P31 wd:Q5 .
  }

  BIND(?role AS ?edgeLabel)

  BIND(
    IF(?role = "Tätigkeit", "FF4136",
      IF(?role = "Ausbildung", "0074D9",
        )AS ?rgb
    )

  SERVICE wikibase:label { bd:serviceParam
wikibase:language "mul,de,en". }
  }
ORDER BY ?tätigkeitLabel ?role
```

Nachgedacht für 33s



du hast aktuell die Personenknoten als „Ziel“ (?place) gebunden – deshalb färbt der

G

+ Stelle irgendeine Frage



SV

ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).


```
}  
  
SERVICE wikibase:label { bd:serviceParam wikibase:language "mul,de,en"  
}  
ORDER BY ?personLabel ?edgeLabel ?targetLabel
```

Jetzt sind die **Zielknoten = ?target** (Berufe/Einrichtungen), farblich je nach Kantenart, und Personen nur die Ausgangsknoten.

     ...

+ Stelle irgendeine Frage



ChatGPT kann Fehler machen. Überprüfe wichtige Informationen. Siehe [Cookie-Voreinstellungen](#).